IERG5050 AI Foundation Models, Systems & Applications
Fall 2025

Agentic AI Applications

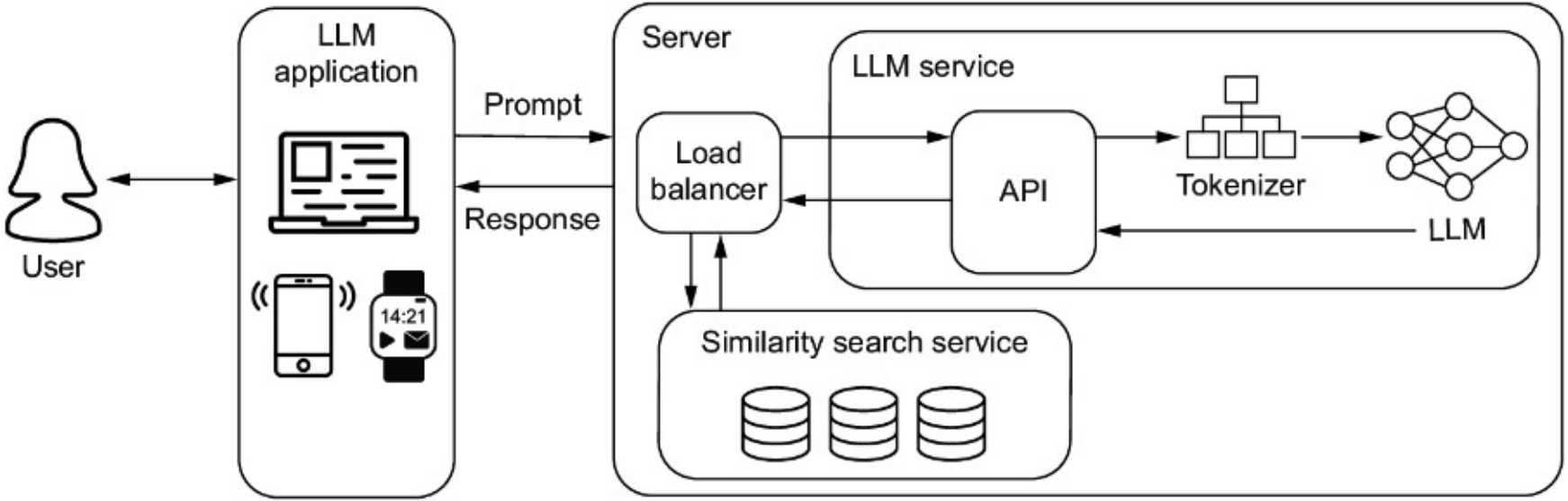
Prof. Wing C. Lau
wclau@ie.cuhk.edu.hk
http://www.ie.cuhk.edu.hk/wclau

Acknowledgements

Many of the slides in this lecture are adapted from the sources below. Copyrights belong to the original authors.

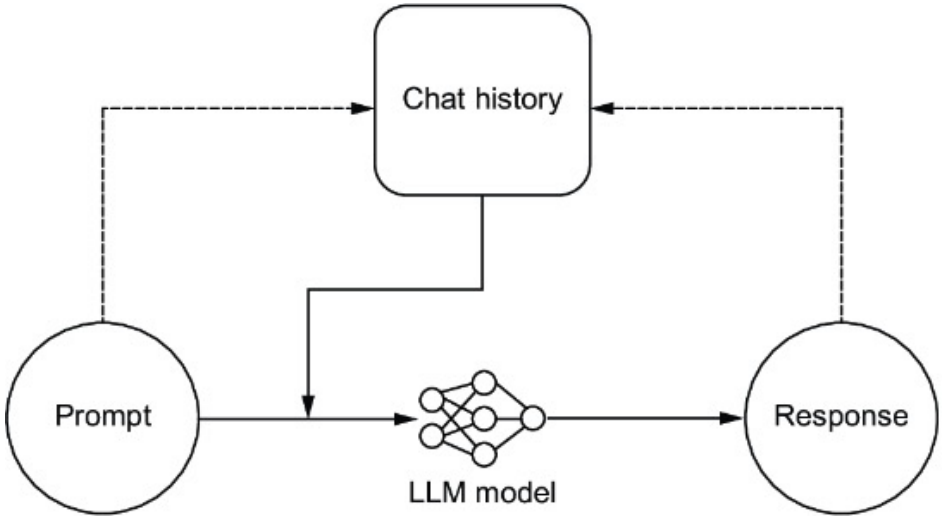
- LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng, https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development/
- LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/
- AI Agents in LangGraph– a short course by Harrison Chase, Rotem Weiss, https://www.deeplearning.ai/short-courses/ai-agents-in-langgraph/
- Building LLM Powered Applications By Valentina Alto, Packt Publishing, May 2024
- AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications
- LLMs in Production by by Christopher Brousseau, Matthew Sharp, Jan 2025, Manning Publications
- Shunyu Yao, “LLM agents: brief history and overview,” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents/assets/llm_agent_history.pdf, https://www.youtube.com/watch?v=RM6ZArd2nVc
- Chi Wang, “Agentic AI Frameworks & AutoGen,” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents-mooc/slides/autogen.pdf, https://www.youtube.com/watch?v=QOdtmCMSQo4
- Jerry Liu, “Building a Multimodal Knowledge Assistant (LlamaIndex),” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents-mooc/slides/MKA.pdf, https://www.youtube.com/watch?v=QOdtmCMSQo4
- Ruslan Salakhutdinov, “Multimodal Autonomous AI Agents,” Guest Lecture for UC Berkeley MOOC on Advanced Large Language Model Agents, Spring 2025, https://rdi.berkeley.edu/adv-llm-agents/slides/ruslan-multimodal.pdf, https://www.youtube.com/live/RPINOYM12RU
- Nicolas Chapados, “AI Agents for Enterprise Workflows,” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents/assets/agentworkflows.pdf, https://www.youtube.com/live/-yf-e-9FyQc
- Ben Mann, “Measuring Agent capabilities and Anthropic’s RSP,” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents/assets/antrsp.pdf, https://www.youtube.com/live/6v2AnWol7oo
- Caiming Xiong, “Multimodal Agents – from Perception to Action,” Guest Lecture for UC Berkeley MOOC on Advanced Large Language Model Agents, Spring 2025, https://rdi.berkeley.edu/llm-agents-mooc/slides/Multimodal_Agent_caiming.pdf, https://www.youtube.com/live/n__Tim8K2IY
- Omar Khattab, “Compound AI Systems & the DSPy Framework,” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents-mooc/slides/dspy_lec.pdf, https://www.youtube.com/live/JEMYuzrKLUw
- Graham Neubig, “Agents for Software Development,” Guest Lecture for UC Berkeley MOOC on Large Language Model Agents, Fall 2024, https://rdi.berkeley.edu/llm-agents-mooc/slides/neubig24softwareagents.pdf, https://www.youtube.com/live/f9L9Fkq-8K4

An LLM-powered Application

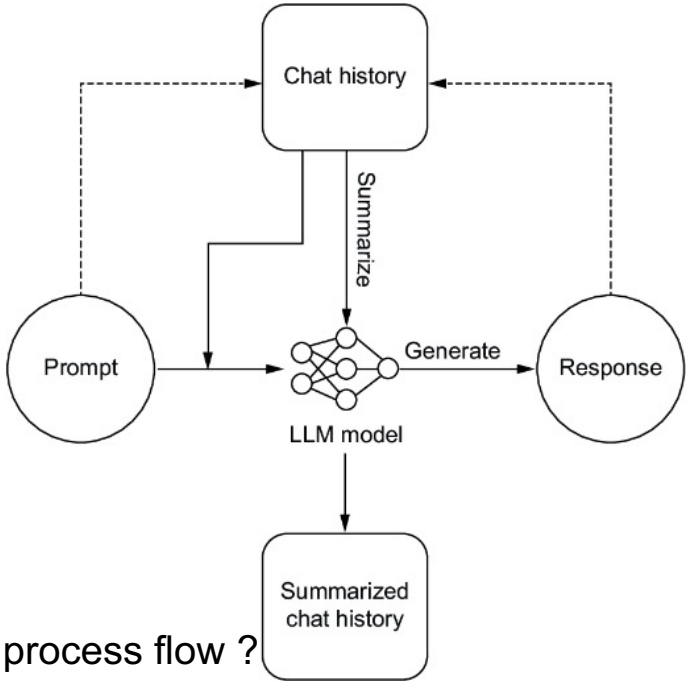


Source: LLMs in Production by By Christopher Brousseau, Matthew Sharp, Jan 2025, Manning Publications

Process-Flow of a Sample LLM-powered Application: a Minimalist ChatBot



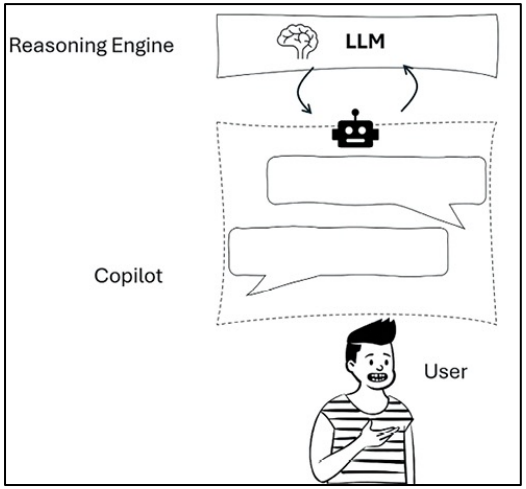
OR



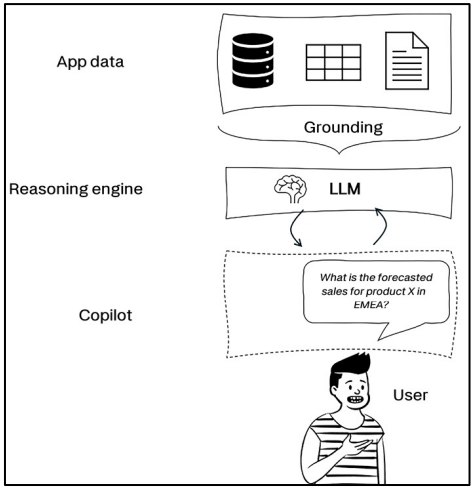
Any other common component needed in the process flow ?

Source: LLMs in Production by By Christopher Brousseau, Matthew Sharp, Jan 2025, Manning Publications

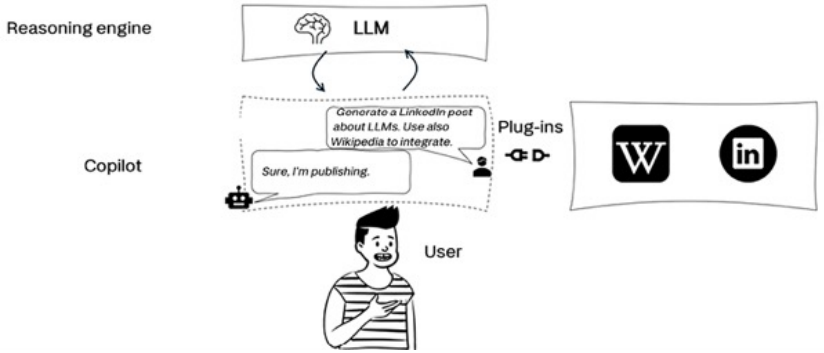
Copilot (aka AI Assistant): An LLM-powered Application



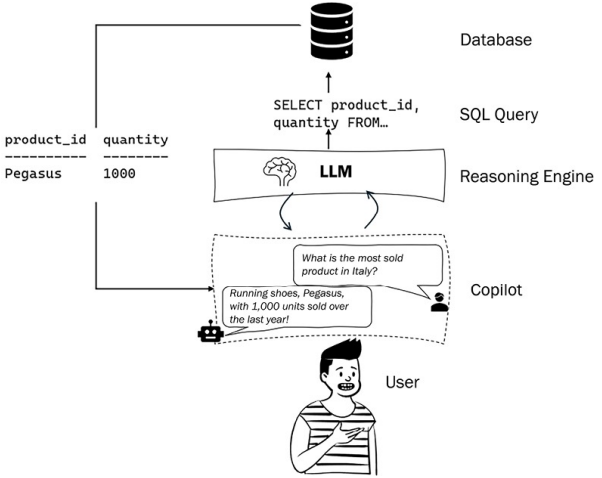
An LLM-powered Copilot



A Copilot w/ grounding

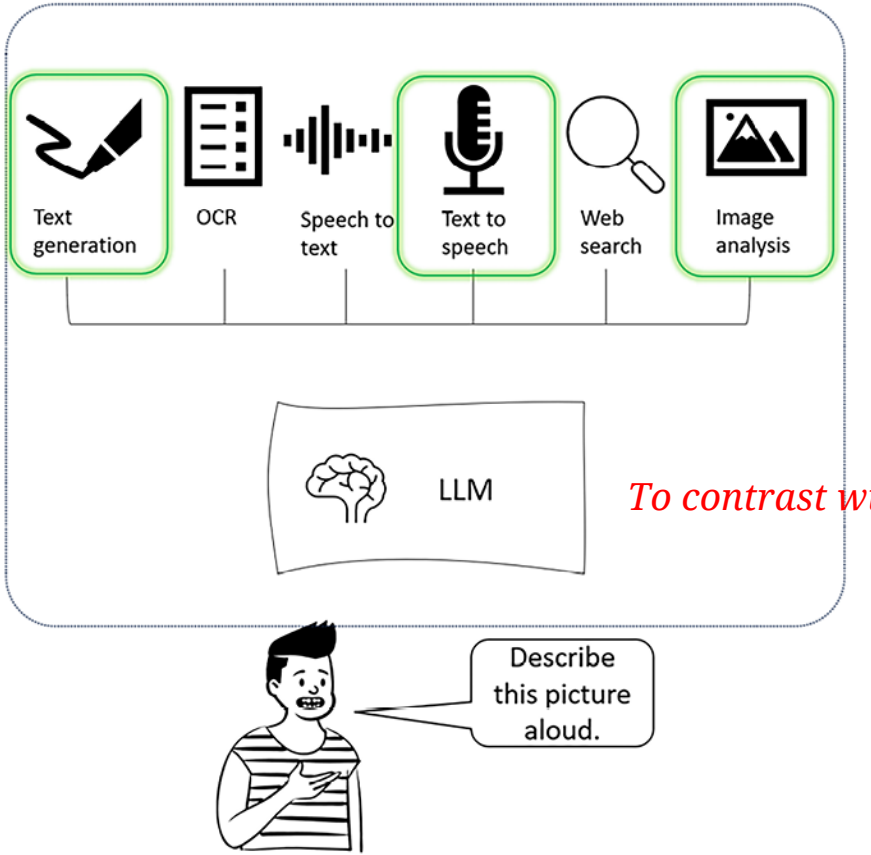


A conversational UI to reduce the gap between the user and the database or ext. tools



Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

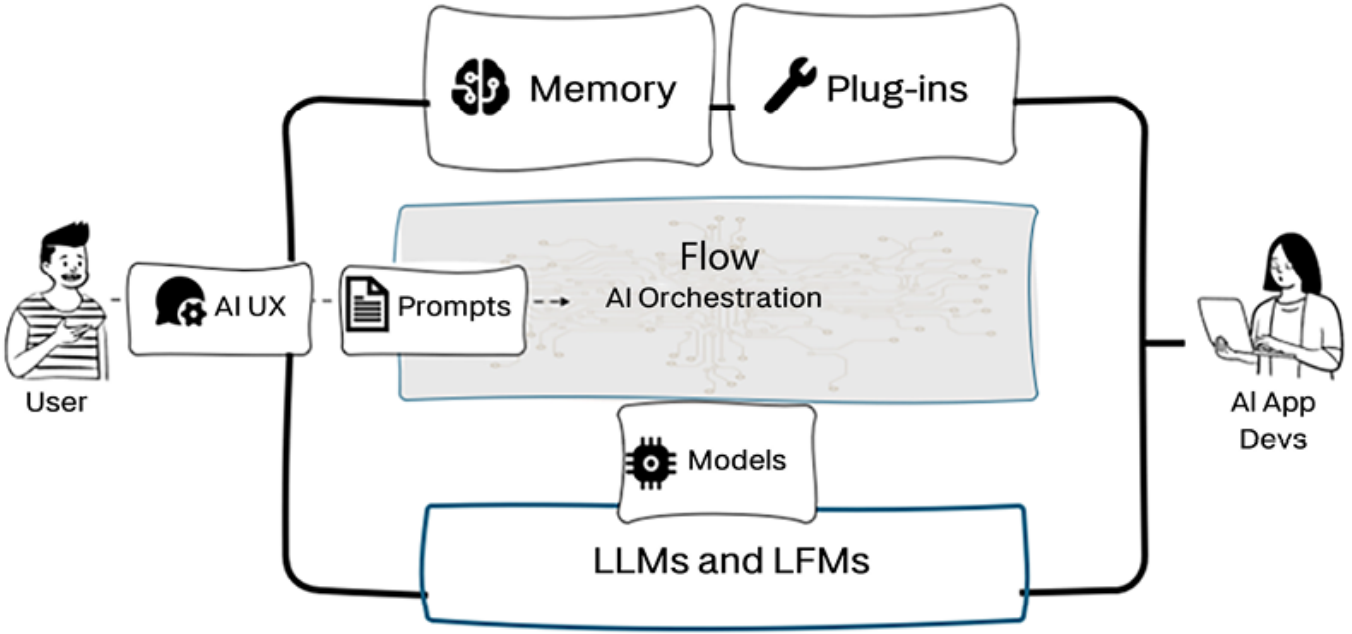
A Multimodal Application via an LLM w/ Single-modal Tools



To contrast with a Multimodal-NATIVE LLM (later)

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

High-level Architecture of an LLM-powered Application



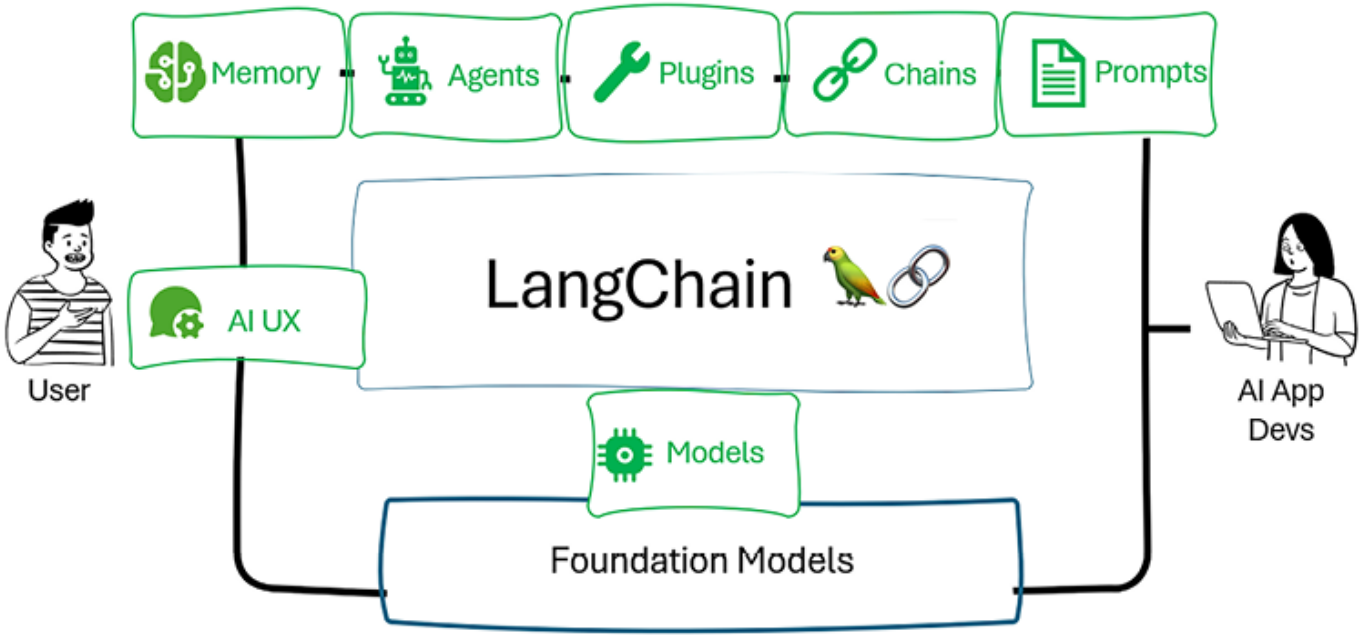
Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

LLM-application/ Agent Development Frameworks

Framework	Best For	Deployment	Scalability	Current Adoption
LangChain & LangGraph	Enterprise AI apps, multi-agent workflows	Self-hosted, Cloud	High – widely used in production	Most adopted
Autogen & Semantic Kernel	Microsoft ecosystem, scalable AI apps	Azure, Self-hosted	High – enterprise-scale	Growing in enterprises
LlamaIndex	AI-powered search, RAG	Cloud, Self-hosted	High – efficient data processing	Strong in AI search
AutoGPT	No-code AI automation, continuous agents	Cloud-first, some self-hosted options	Medium – automation focused	Popular for prototyping
CrewAI	Workflow-based multi-agent AI apps	Cloud, Self-hosted	Medium – still early-stage	Fast-growing
PydanticAI	FastAPI & Pydantic-based AI apps	Self-hosted	Low – lightweight framework	Niche adoption
Spring AI	Java-based AI applications	Self-hosted, Cloud	Medium – depends on Java ecosystem	Popular in Java world
Haystack	LLM-powered search & RAG	Self-hosted, Cloud	High – built for production	Strong in search applications

Source: https://medium.com/@varmalearn/a-casual-guide-to-the-top-agentic-ai-frameworks-in-2025-9299dd94e034

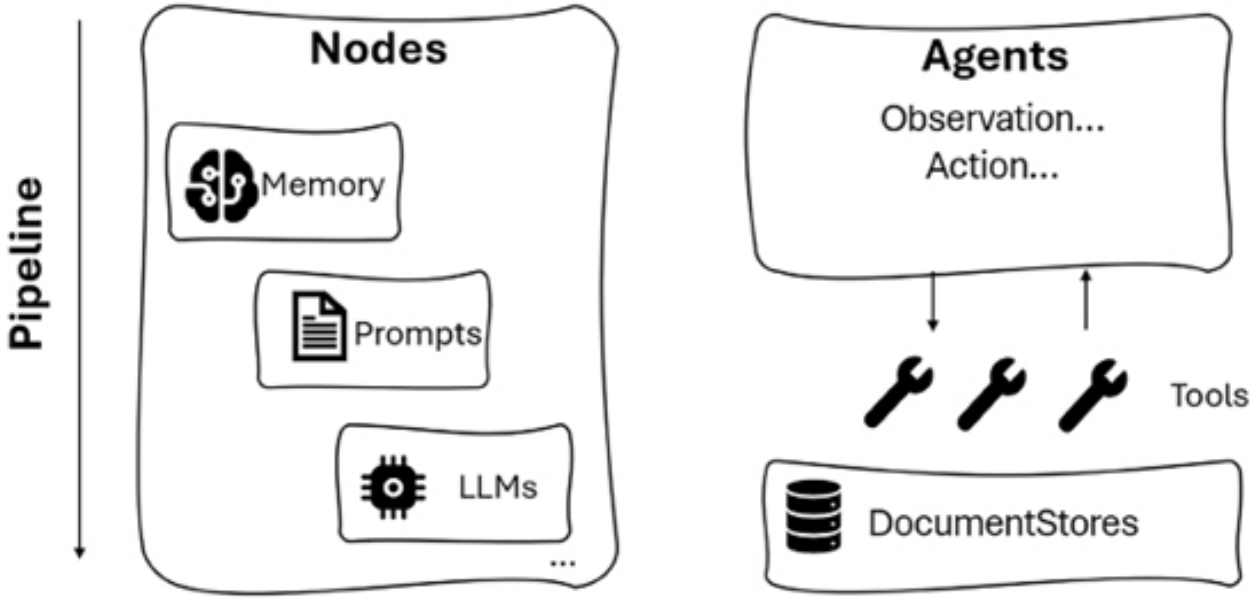
LangChain: a framework for building LLM-based Applications



Components of LangChain

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

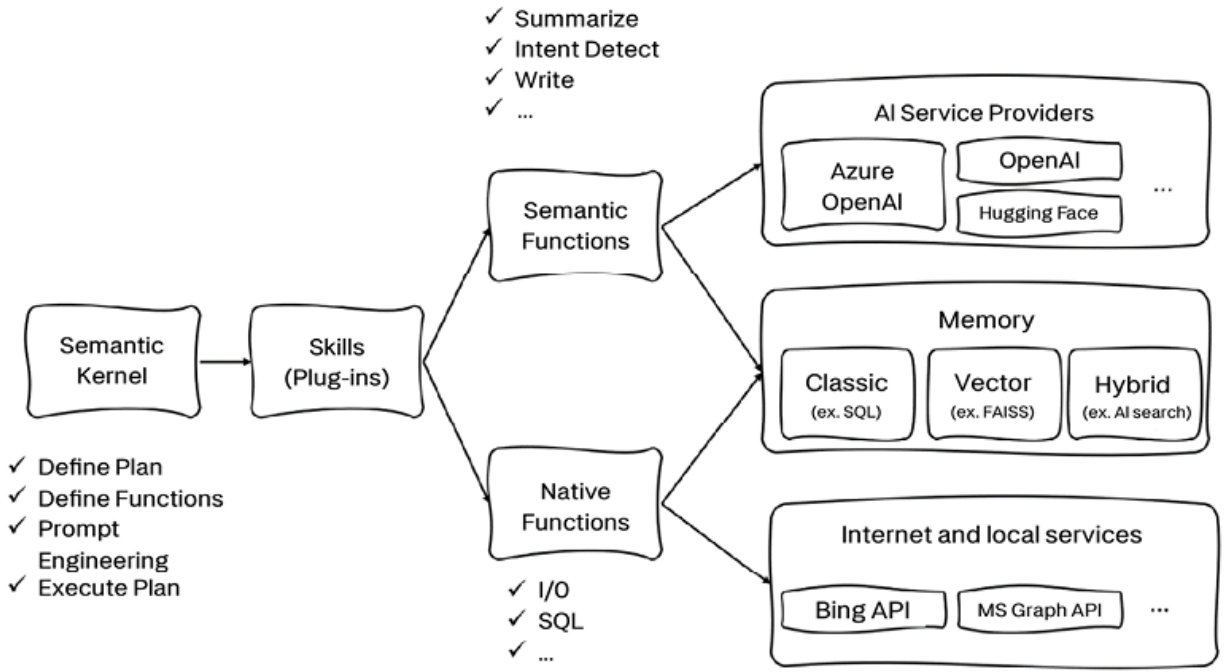
Haystack: a framework for building LLM-based Applications



Haystack's components

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

Semantic Kernel: a framework for building LLM-based Applications



Anatomy of Semantic Kernel

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

LangChain vs. Haystack vs. Semantic Kernel

Feature	LangChain	Haystack	Semantic Kernel
LLM support	Proprietary and open-source	Proprietary and open source	Proprietary and open source
Supported languages	Python and JS/TS	Python	C#, Java, and Python
Process orchestration	Chains	Pipelines of nodes	Pipelines of functions
Deployment	No REST API	REST API	No REST API
Feature	**LangChain**	**Haystack**	**Semantic Kernel**

Table 2.1: Comparisons among the three AI orchestrators

More on Semantic Kernel as well as other Agentic AI frameworks later…

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

Developing LLM-based Applications with LangChain

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

LangChain Overview

Open-source development framework for LLM applications

Python and JavaScript (TypeScript) packages

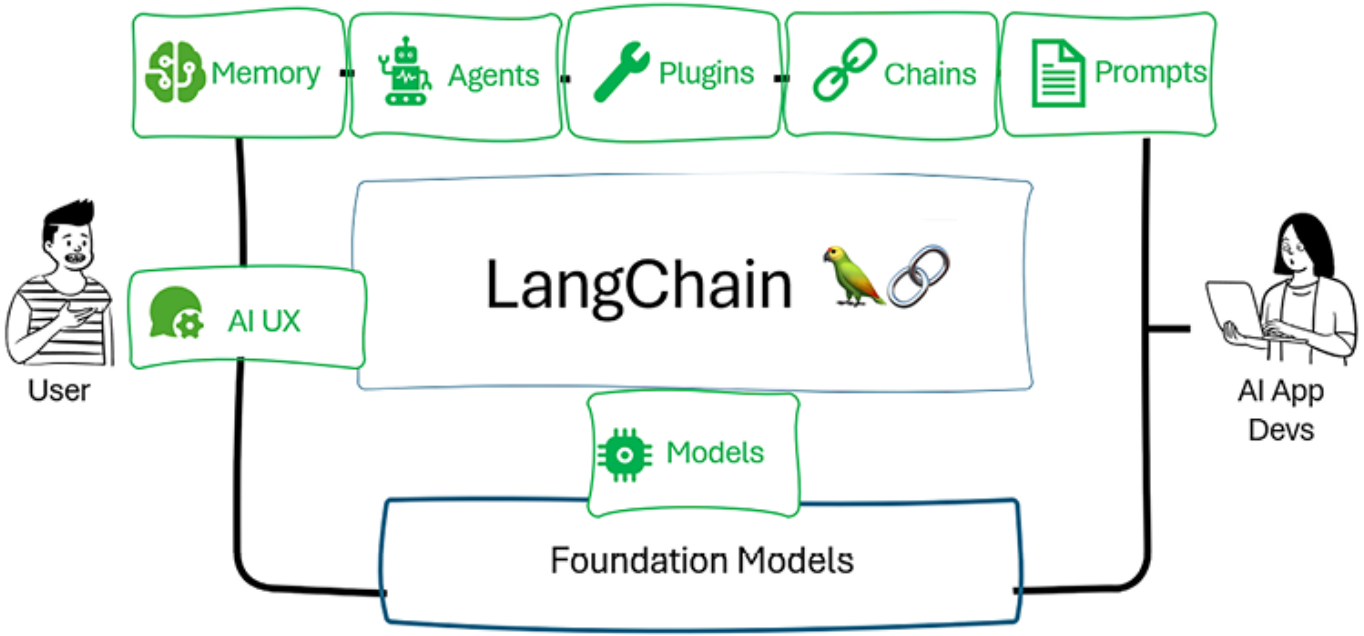
Focused on composition and modularity

Key value adds:

1. Modular components
2. Use cases: Common ways to combine components

Source: LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng, https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development/

LangChain: a framework for building LLM-based Applications



Components of LangChain

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

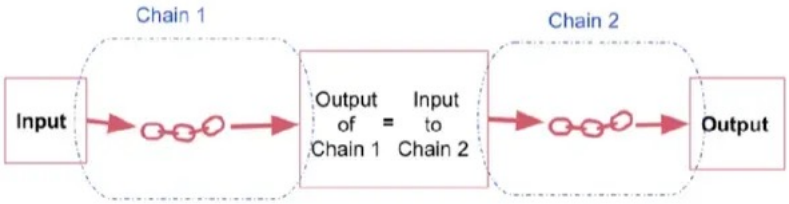
Components of LangChain

- Models
  - LLMs: 20+ integrations
  - Chat Models
  - Text Embedding Models: 10+ integrations
- Prompts
  - Prompt Templates
  - Output Parsers: 5+ implementations
    - Retry/fixing logic
  - Example Selectors: 5+ implementations
- Indexes
  - Document Loaders: 50+ implementations
  - Text Splitters: 10+ implementations
  - Vector stores: 10+ integrations
  - Retrievers: 5+ integrations/implementations
- Chains
  - Prompt + LLM + Output parsing
  - Can be used as building blocks for longer chains
  - More application specific chains: 20+ types
- Agents
  - Agent Types: 5+ types
    - Algorithms for getting LLMs to use tools
  - Agent Toolkits: 10+ implementations
    - Agents armed with specific tools for a specific application

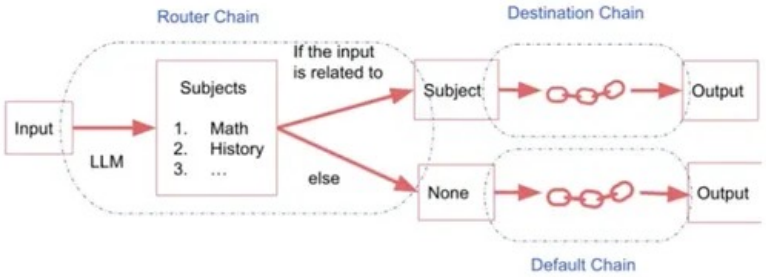
Source: LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng, https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development/

Process Flow Orchestration with “Chains”

Simple Sequential Chain: One i/p, One o/p

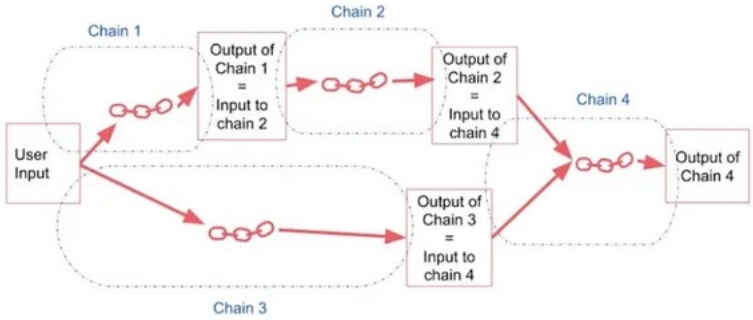


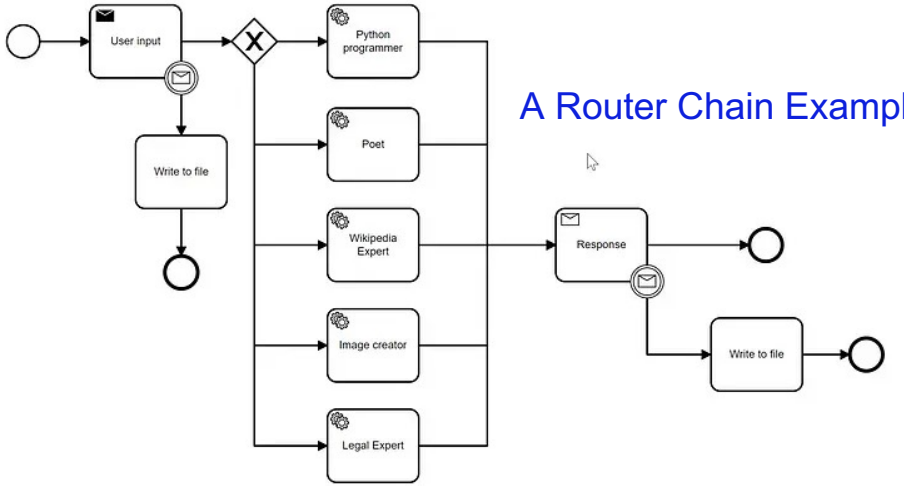
Router Chain:



A Chain is sequence of calls. Those calls can be to LLMs, External Tools, or Data Processing steps

Sequential Chain: Multiple i/p, Multiple o/p

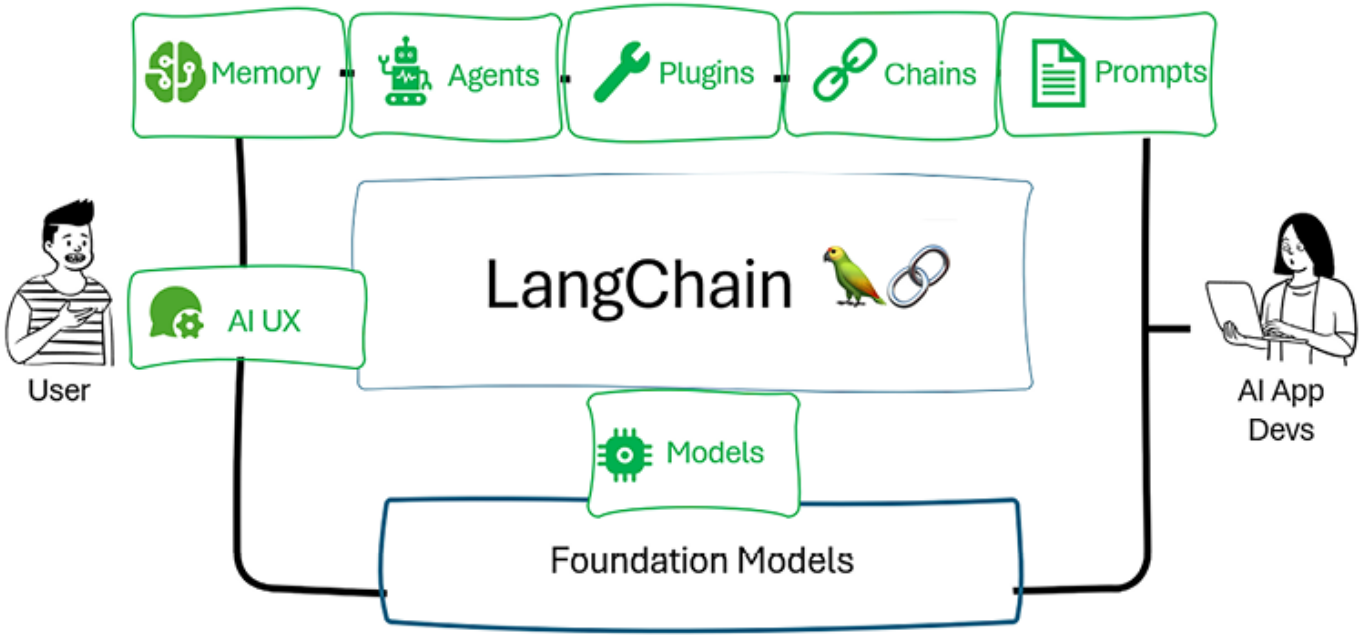




A Router Chain Example

Source: LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng.

LangChain: a framework for building LLM-based Applications



Components of LangChain

Source: Building LLM Powered Applications by Valentina Alto, Packt Publishing, May 2024

LangChain Memory Types

ConversationBufferMemory

- This memory allows for storing of messages and then extracts the messages in a variable.

ConversationBufferWindowMemory

- This memory keeps a list of the interactions of the conversation over time. It only uses the last K interactions.

ConversationTokenBufferMemory

- This memory keeps a buffer of recent interactions in memory, and uses token length rather than number of interactions to determine when to flush interactions.

ConversationSummaryMemory

- This memory creates a summary of the conversation over time.

Source: LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng, https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development/

LangChain Memory Types

Vector data memory

- Stores text (from conversation or elsewhere) in a vector database and retrieves the most relevant blocks of text.

Entity memories

- Using an LLM, it remembers details about specific entities.

You can also use multiple memories at one time.
E.g., Conversation memory + Entity memory to recall individuals.

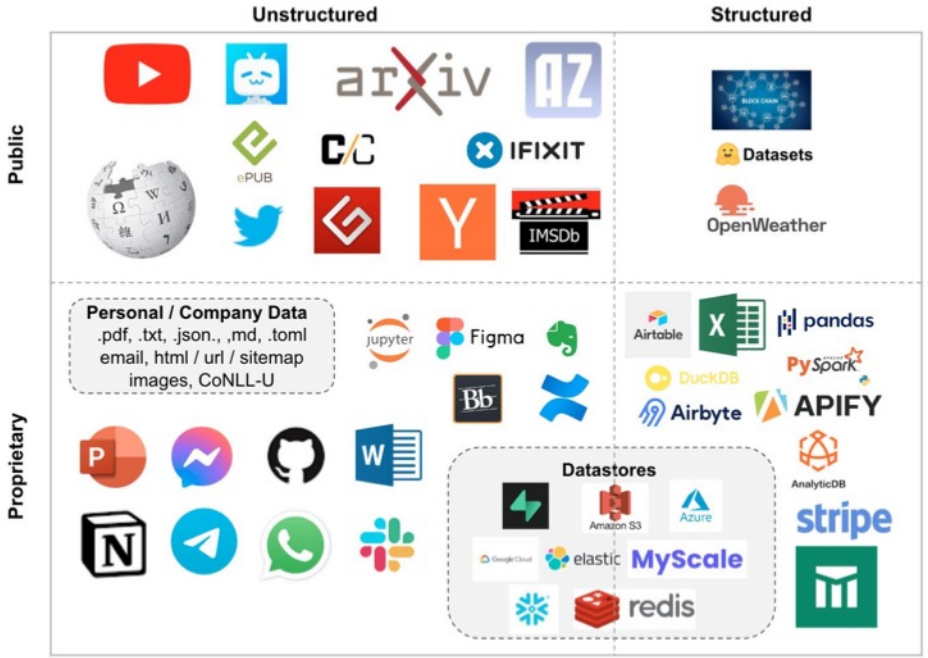
You can also store the conversation in a conventional database (such as key-value store or SQL)

Source: LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng, https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development/

Document Loading with LangChain

- Loaders deal with the specifics of accessing and converting data
  - Accessing
    - Web Sites
    - Data Bases
    - YouTube
    - arXiv
    - …
  - Data Types
    - PDF
    - HTML
    - JSON
    - Word, PowerPoint…
- Returns a list of `Document` objects:

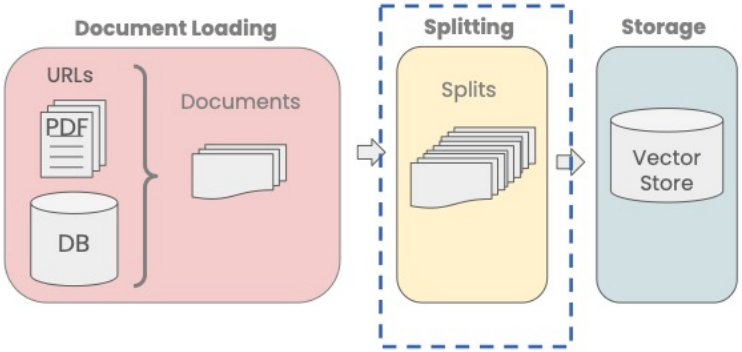
```
[
Document(page_content='MachineLearning-Lecture01 \nInstructor (Andrew Ng): Okay. Good morning. Welcome to CS229....',
metadata={'source': 'docs/cs229_lectures/MachineLearning-Lecture01.pdf', 'page': 0})
...
 Document(page_content='[End of Audio]  \nDuration: 69 minutes  ',
 metadata={'source': 'docs/cs229_lectures/MachineLearning-Lecture01.pdf', 'page': 21})
]
```



Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Document Splitting with LangChain

- Splitting Documents into smaller chunks
  - Retaining meaningful relationships!



...
on this model. The Toyota Camry has a head-snapping
80 HP and an eight-speed automatic transmission that will
...

Chunk 1: on this model. The Toyota Camry has a head-snapping

Chunk 2: 80 HP and an eight-speed automatic transmission that will

Question: What are the specifications on the Camry?

Example Splitter

```
langchain.text_splitter.CharacterTextSplitter(
    separator: str = "\n\n"
    chunk_size=4000,
    chunk_overlap=200,
    length_function=<builtin function len>,
)
```

Methods:

create_documents() - Create documents from a list of texts.

split_documents() - Split documents.

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

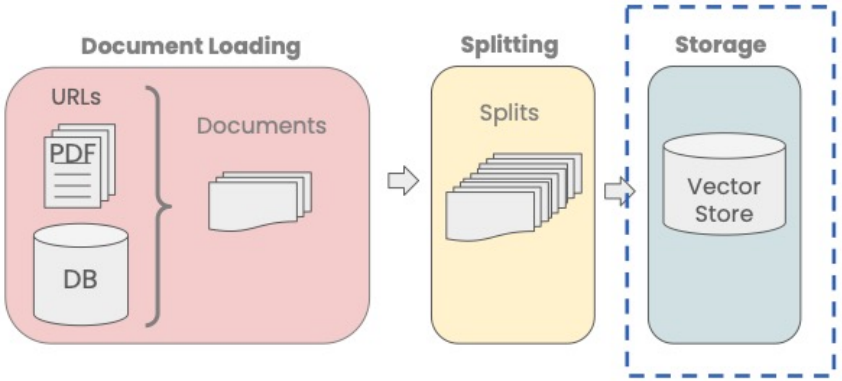
Different Types of Splitters in LangChain

langchain.text_splitter.

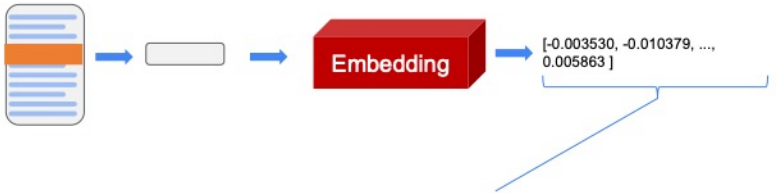
- **CharacterTextSplitter()-** Implementation of splitting text that looks at characters.
- **MarkdownHeaderTextSplitter**() - Implementation of splitting markdown files based on specified headers.
- **TokenTextSplitter**() - Implementation of splitting text that looks at tokens.
- **SentenceTransformersTokenTextSplitter**() - Implementation of splitting text that looks at tokens.
- ***RecursiveCharacterTextSplitter()*** **- Implementation of splitting text that looks at characters. Recursively tries to split by different characters to find one that works.**
- **Language() –** for CPP, Python, Ruby, Markdown etc
- **NLTKTextSplitter**() - Implementation of splitting text that looks at sentences using NLTK (Natural Language Tool Kit)
- **SpacyTextSplitter() -** Implementation of splitting text that looks at sentences using Spacy

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Support of Vector Store in LangChain

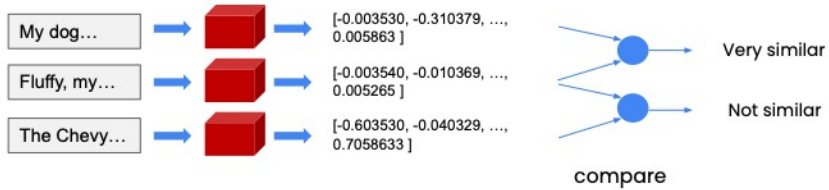


Embeddings



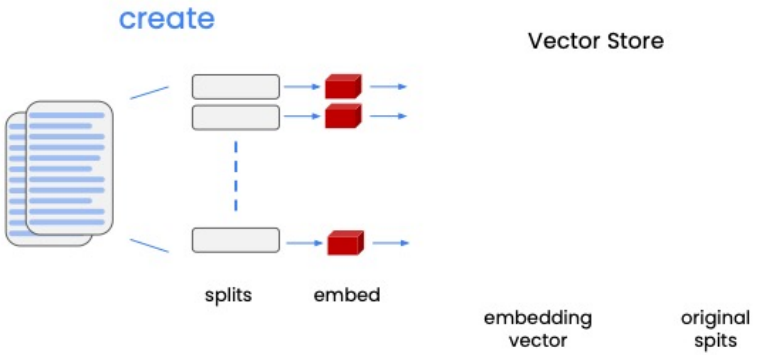
- Embedding vector captures content/meaning
- Text with similar content will have similar vectors

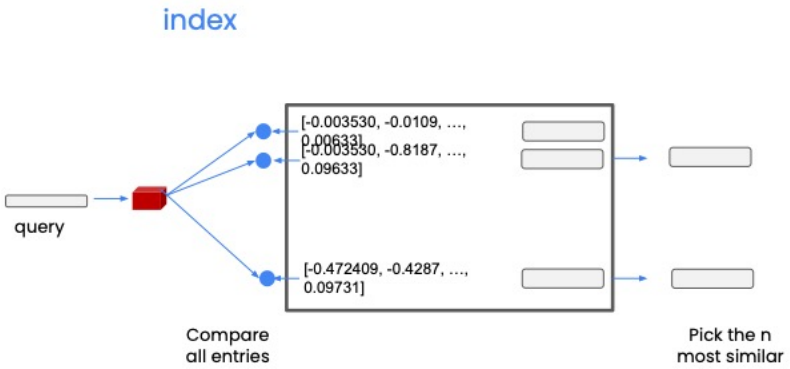
1) My dog Rover likes to chase squirrels.
2) Fluffy, my cat, refuses to eat from a can.
3) The Chevy Bolt accelerates to 60 mph in 6.7 seconds.

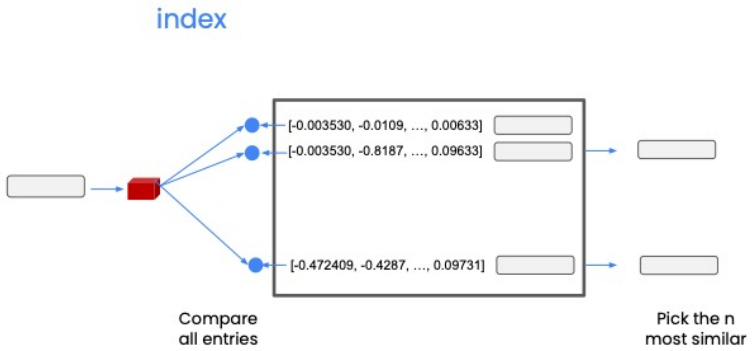


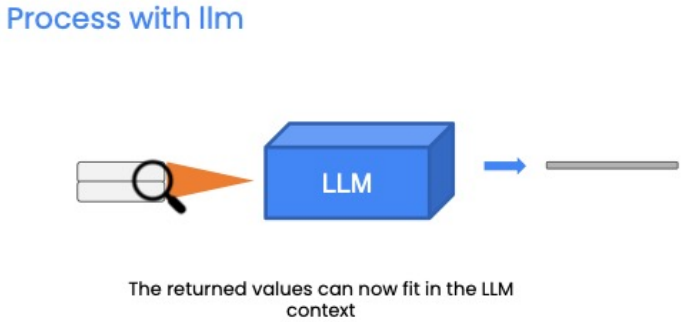
Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Basic RAG support w/ Vector Store in LangChain



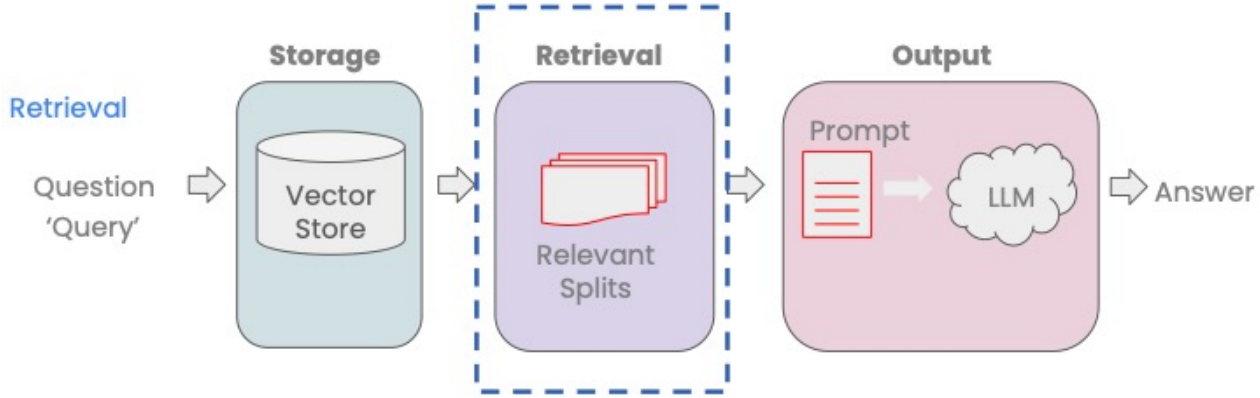






Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Retrieval Support in LangChain

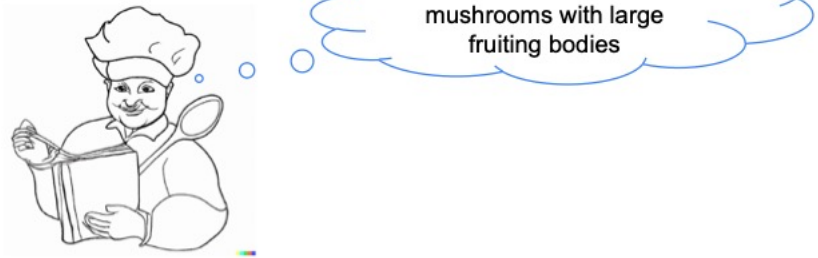


- Accessing/indexing the data in the vector store
  - Basic semantic similarity
  - Maximum marginal relevance
  - Including Metadata
- LLM Aided Retrieval

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

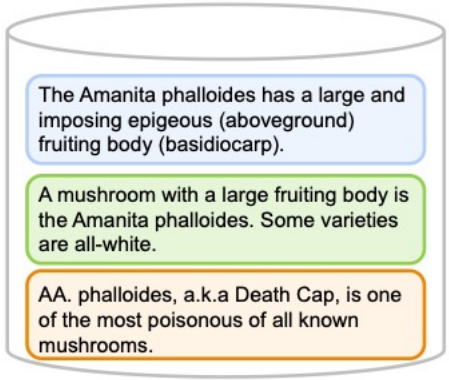
Retrieval based on Maximum Marginal Relevance (MMR)

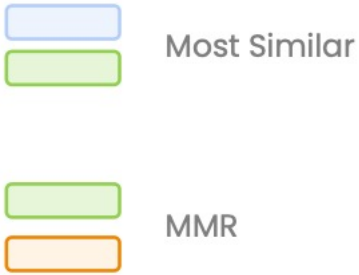
- You may not always want to choose the most similar responses

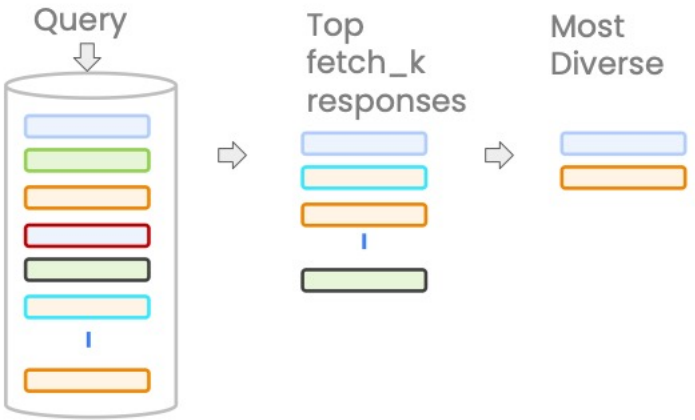


MMR algorithm

- Query the Vector Store
- Choose the `fetch_k` most similar responses
- Within those responses choose the `k` most diverse



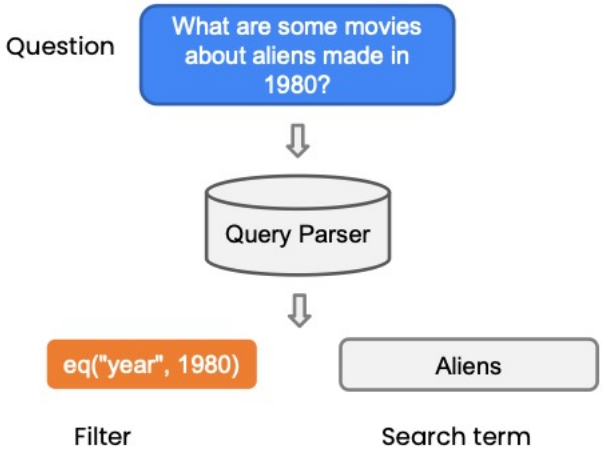


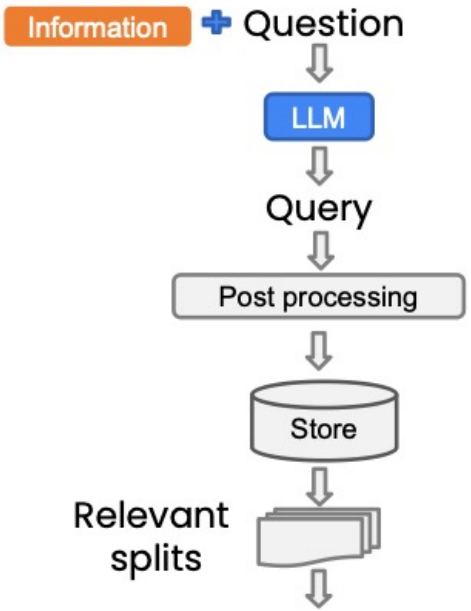


Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

LLM-aided Retrieval

- There are several situations where the Query applied to the DB is more than just the Question asked.
- One is SelfQuery, where we use an LLM to convert the user question into a query





Self-query

Information: Query format

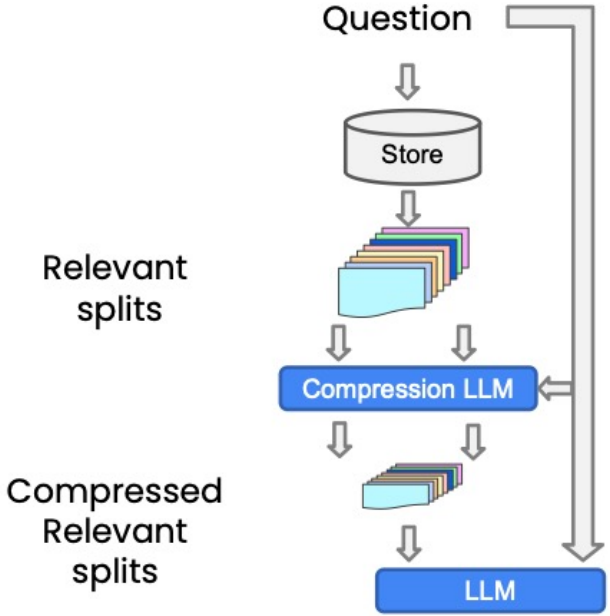
Query: Question
Filter: eq["section", "testing"]

Query parser

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

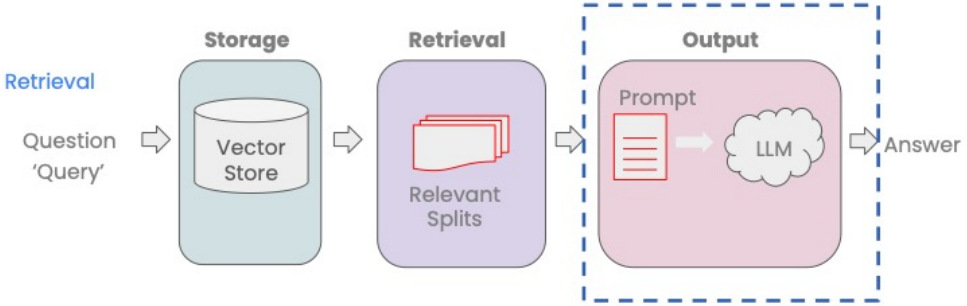
Compression Support

- Increase the number of results you can put in the context by shrinking the responses to only the relevant information.

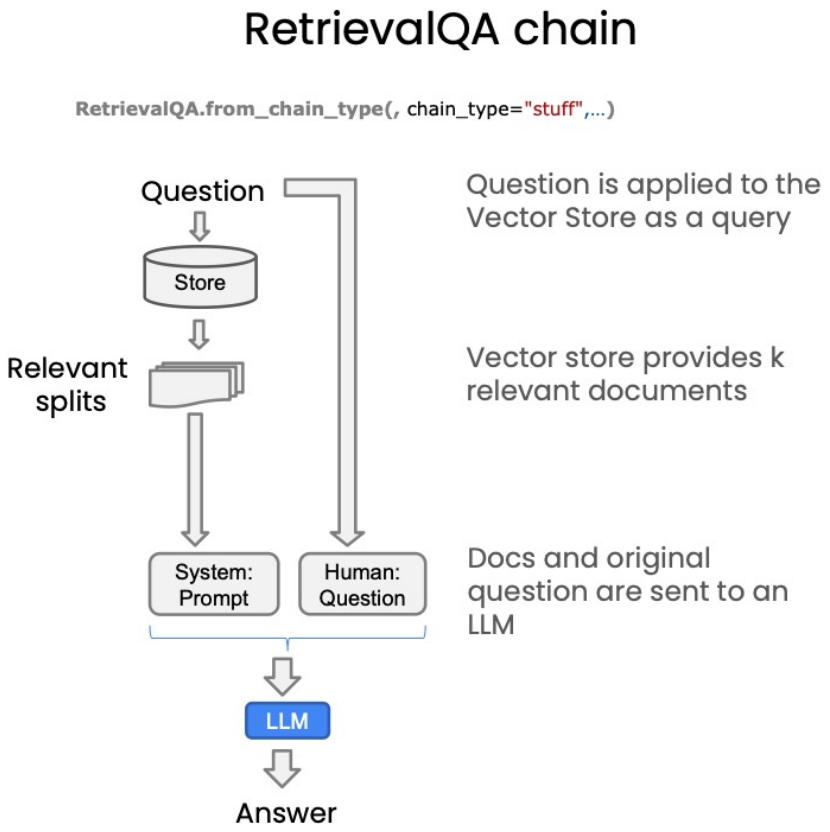


Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Q&A Support via Retrieval



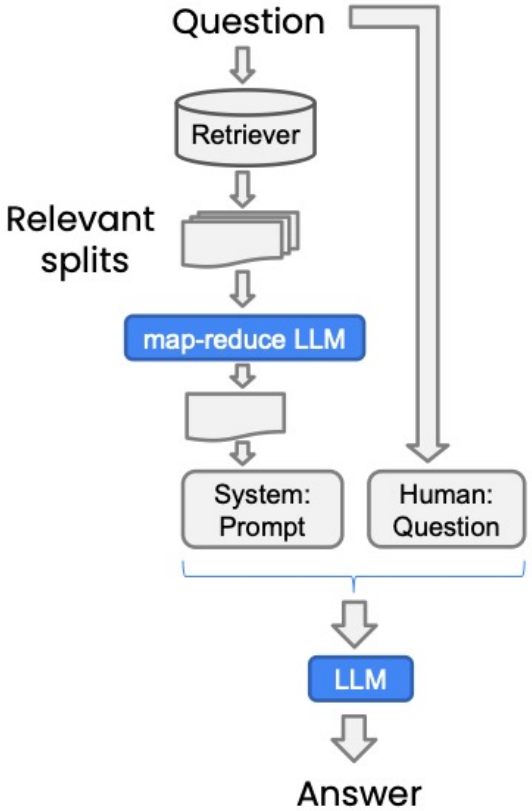
- Multiple relevant documents have been retrieved from the vector store
- Potentially compress the relevant splits to fit into the LLM context
- Send the information along with our question to an LLM to select and format an answer



Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Retrieval Chain with LLM Selection

```
RetrievalQA.from_chain_type(, chain_type="map_reduce",...)
```



You many have too many docs to fit into an LLM context. The solution is to use an LLM to select the 'most relevant' information

3 additional methods

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Facilitating Q&A over Documents with LangChain

Stuff method

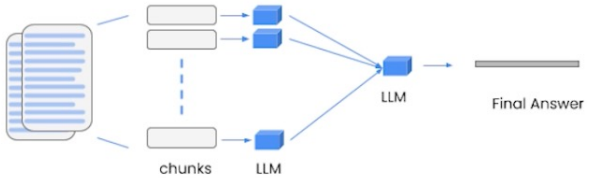
Stuffing is the simplest method. You simply stuff all data into the prompt as context to pass to the language model.

Pros: It makes a single call to the LLM. The LLM has access to all the data at once.

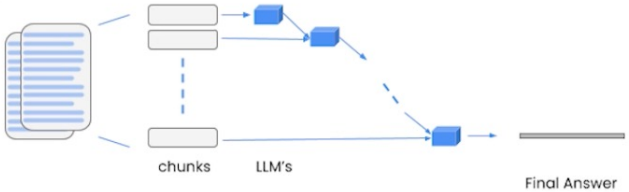
Cons: LLMs have a context length, and for large documents or many documents this will not work as it will result in a prompt larger than the context length.

3 additional methods

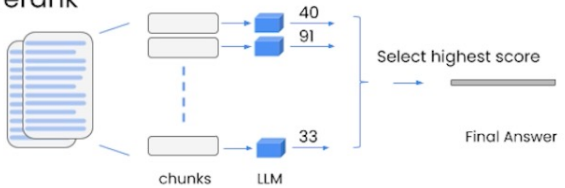
1. Map_reduce



2. Refine



3. Map_rerank



Source: LangChain for LLM Application Development – a short course by Harrison Chase, Andrew Ng, https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development/

Agent Support in LangChain

Agent refers to the idea of using large language models as reasoning engine to determine which actions to take and in what order.

An action can be using a tool and observing its output and deciding what to return to the user.

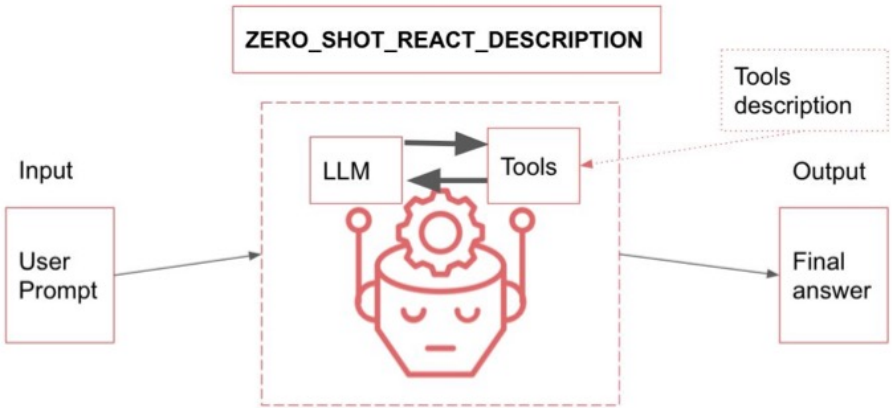
```
from langchain.agents import
initialize_agent, AgentType

agent = initialize_agent(tools, llm,
      agent=AgentType.ZERO_SHOT_REACT_DESCRI
PTION,      verbose=True)
```

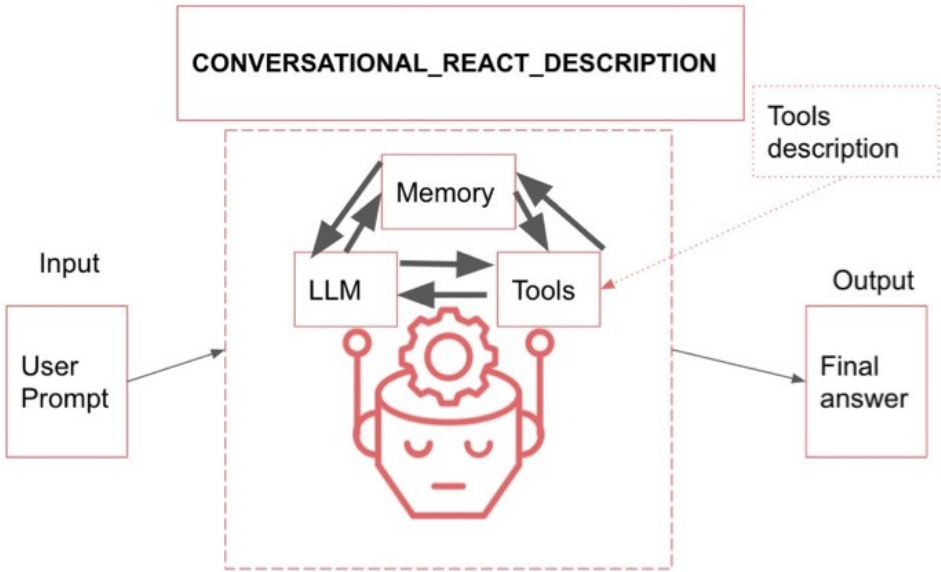
To construct a minimalist Agent, we need:

- PromptTemplate: this is responsible for taking the user input and previous steps and constructing a prompt to send to the language model
- Language Model: this takes the prompt constructed by the PromptTemplate and returns some output
- Output Parser: this takes the output of the Language Model and parses it into an AgentAction or AgentFinish object.

Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

More Agent Support in LangChain





Source: LangChain Chat with your Data – a short course by Harrison Chase, https://www.deeplearning.ai/short-courses/langchain-chat-with-your-data/

Enpowering LLM-based Applications with External Actions

How ChatGPT interacts w/ the Outside World via Plugins

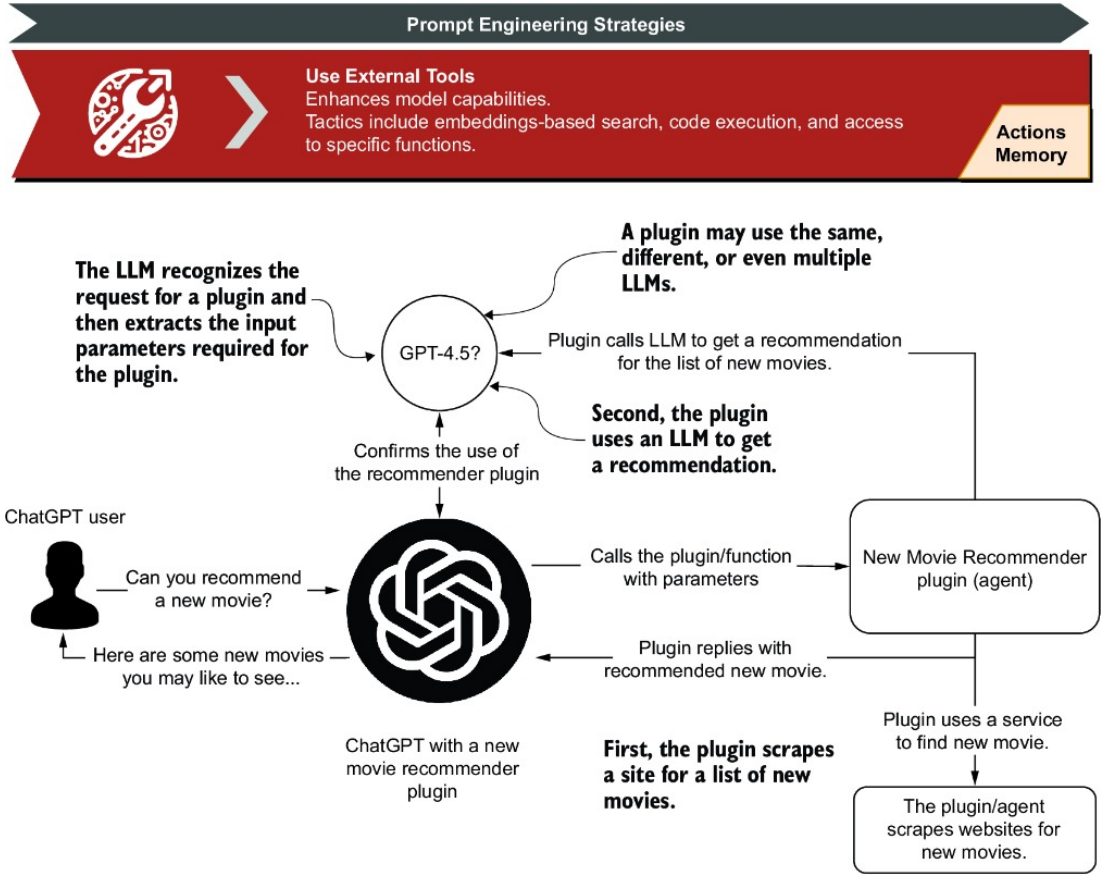
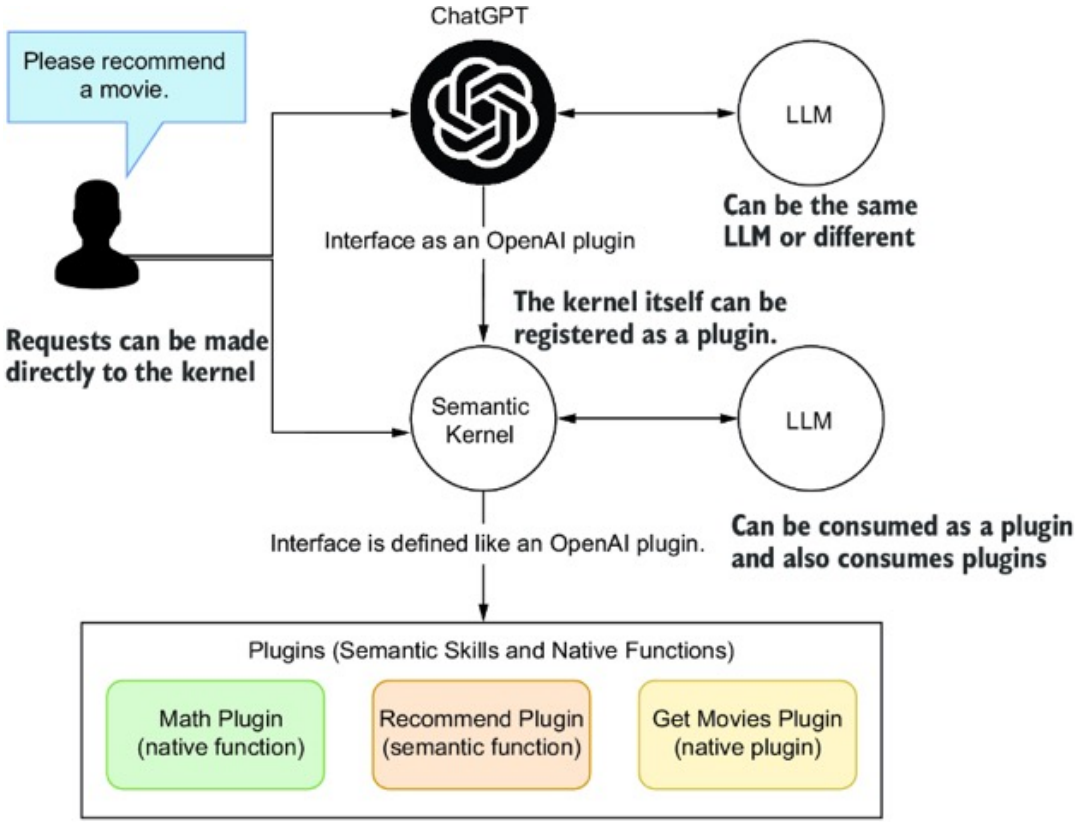


Figure 5.1 How a ChatGPT plugin operates and how plugins and other external tools (e.g., APIs) align with the Use External Tools prompt engineering strategy

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Microsoft's Semantic Kernel as a Plugin for ChatGPT



Semantic Kernel (SK) is another open source project from Microsoft intended to help build AI applications. At its core, the project is best used to define actions, or what the platform calls semantic plugins, which are wrappers for skills and functions.

This figure shows how the SK can be used as a plugin and a consumer of OpenAI plugins.

The SK relies on the OpenAI plugin definition to define a plugin. That way, it can consume and publish itself or other plugins to other systems.

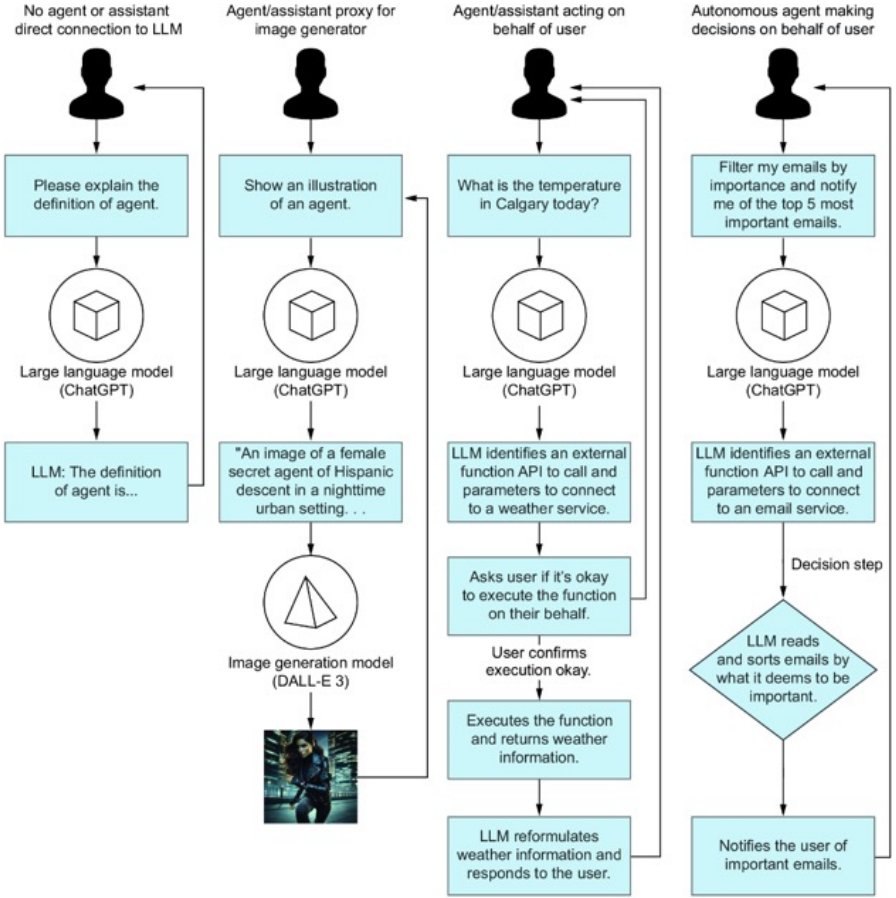
An OpenAI plugin definition maps precisely to the function definitions. This means that SK is the orchestrator of API tool calls, aka plugins.

That also means that SK can help organize multiple plugins with a chat interface or an agent.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

From LLM-powered Application to AI Agent

From “LLM-powered Application” to “AI Agent”



Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

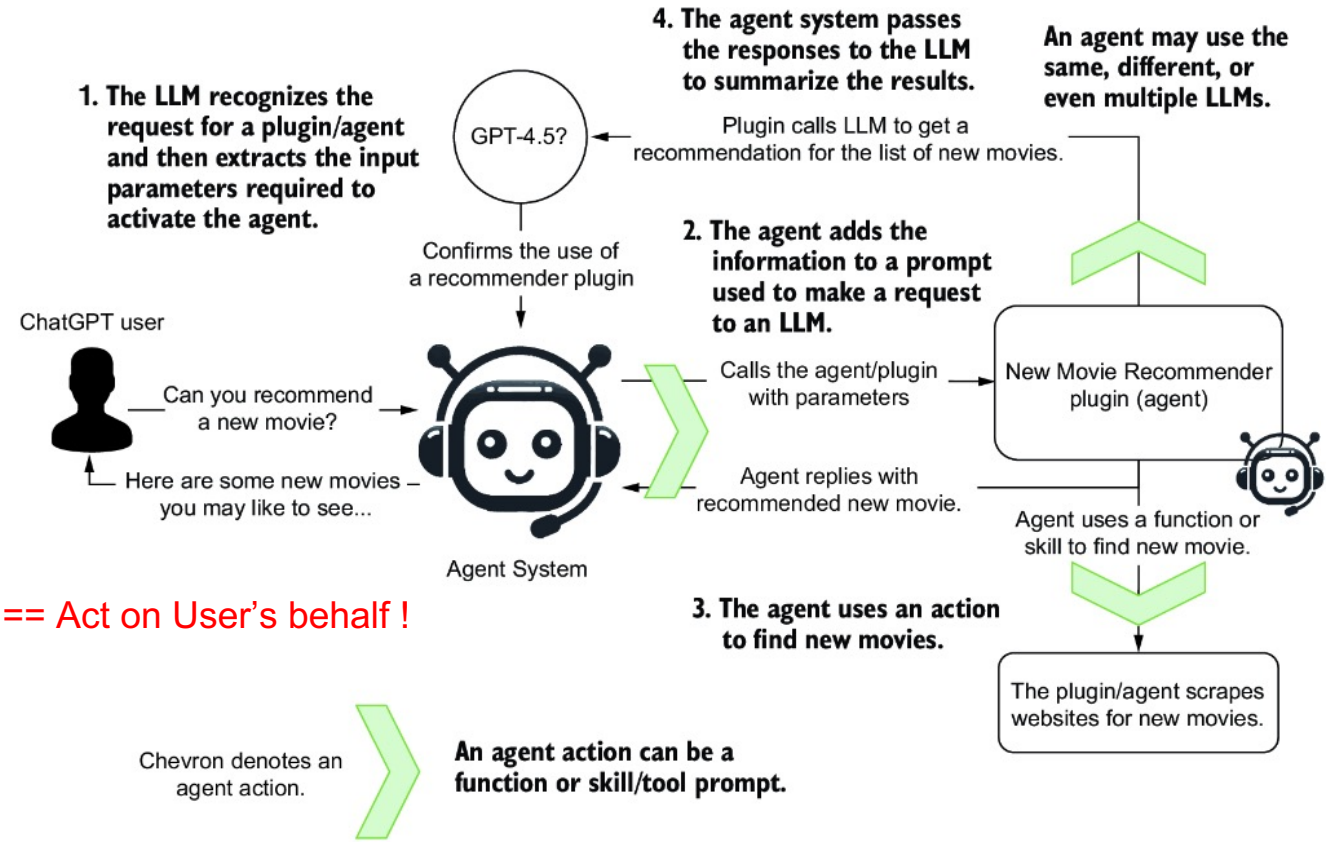
Agent == Act on “YOUR” behalf at various level of autonomy !

“YOUR” can be == Users

“YOUR” can also be an Ext. App

The differences between the LLM interactions from direct action compared to using proxy agents, agents, and autonomous agents

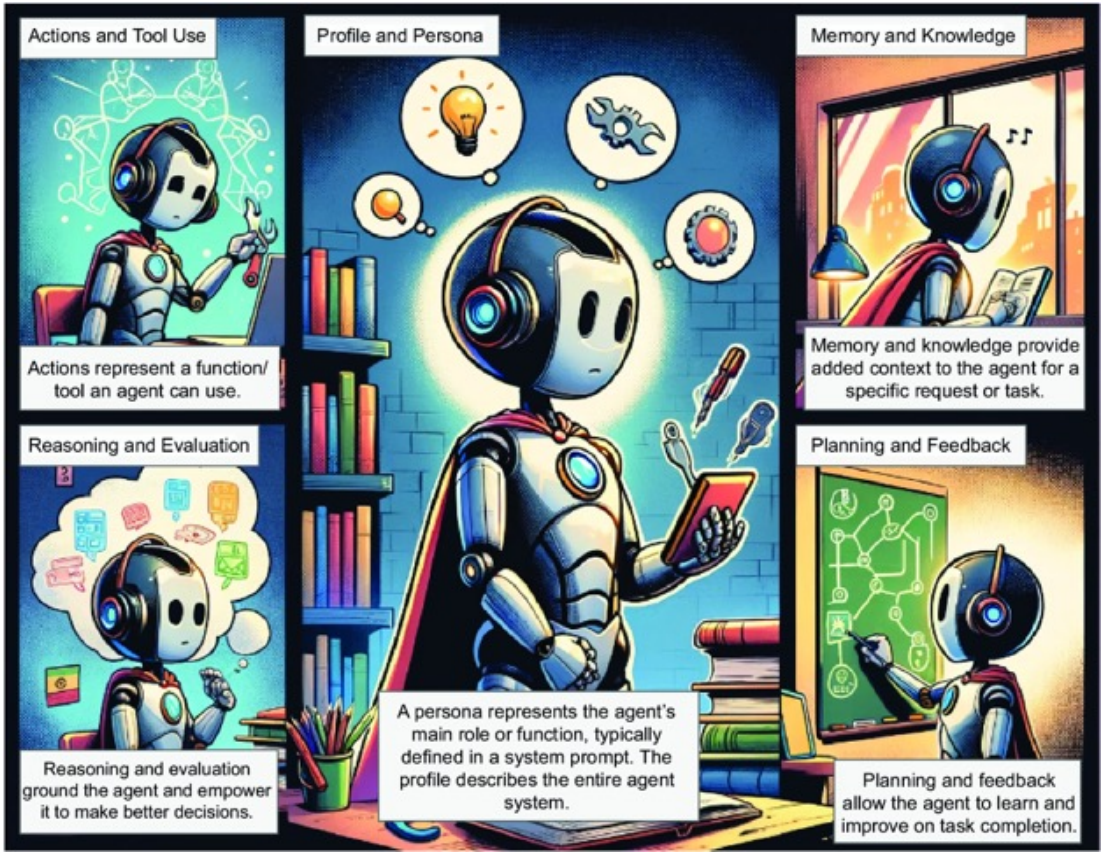
How an Agent uses Actions to perform External Tasks



Agent == Act on User's behalf !

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Key Functional Components of a Single-Agent System



Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

In-Depth Look at the Key Functional Components of an Agent

The Profile and Persona Component of an Agent

Profile and Persona

Profile Contents

Persona: Role, i.e., coder or tester
Demographics: Sex, age, background

Profile Generation

Handcrafted: Manually designed by humans
LLM generated: Directed by human prompts
Data generated: Constructed from data personas

Agent persona: We'll understand how to clearly define the persona, specifying their role and characteristics to guide the agent effectively.

Agent role and demographics: We'll see how relevant demographic and role details can provide agent context, such as age, gender, or background, for a more relevant interaction.

Human vs. AI assistance for persona generation: We'll highlight the role of human involvement in persona generation, whether it's entirely human driven or assisted by LLMs or other agents.

Innovative persona techniques: Prompts generated through data or other novel approaches such as evolutionary algorithms to enhance agent capabilities.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Agent Action and Tool Use

Action targets: We'll learn the importance of defining action targets, whether for task completion, exploration, or communication, to clarify the agent's objectives.

Action space and impact: We'll learn the significance of understanding how actions affect task completion and their effect on the agent's environment, internal states, and self-knowledge.

Action generation methods: We'll see the various ways actions can be generated, such as manual creation, memory recollection, or plan following, to illustrate the diversity of agent behaviors.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Agent Memory and Knowledge

Memory and Knowledge

Retrieval Structure
- Unified
- Hybrid

Retrieval Formats
- Language
- Databases
- Embeddings
- Lists

Retrieval Operation
- Augmentation
- Semantic Extraction
- Compression

Retrieval structure variety: We'll learn about the diverse memory structures agents can employ, including unified and hybrid approaches, enabling flexibility in information storage.

Retrieval formats: We'll explore the various data sources for memory, such as language (e.g., PDF documents), databases (relational, object, or document), and embeddings, offering a rich pool of information to draw upon.

Semantic similarity: We'll learn how embeddings enable semantic similarity searches, facilitating efficient retrieval of relevant data and enhancing the agent's decision-making capabilities.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Reasoning and Evaluation Functions of an Agent

Reasoning and Evaluation

Reasoning
- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Chain of thought
- Tree of thought
- Skeleton of thought

Evaluation
- Self-consistency
- Prompt chaining

Reasoning enables the agent to self-reflect and internally reason out the completion of a task or tasks.

Evaluation provides the basis for an agent's self-reflection on working through and upon task completion.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Planning and Feedback Functions of an Agent

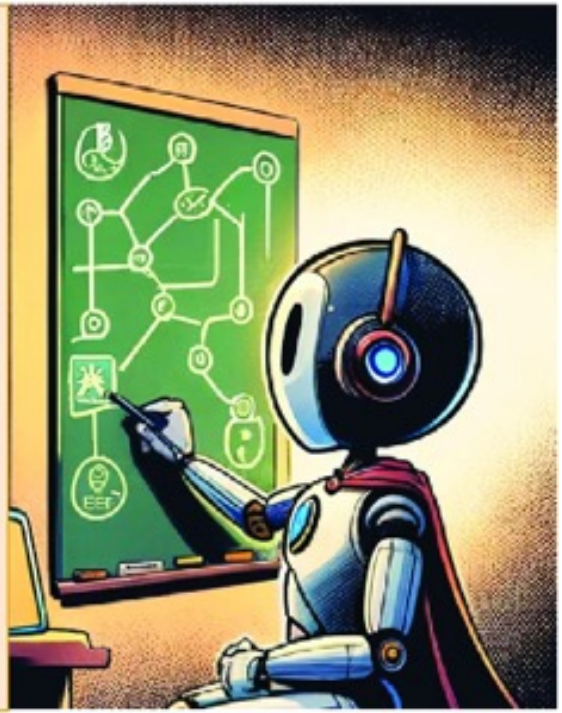
We'll look at various planning strategies with and without feedback—from basic and sequential planners to automatic tool use with reasoning.

Feedback may come from a variety of sources, such as environmental, human, and an LLM via various constructive feedback patterns.

Planning and Feedback

Planning without feedback (autonomous)
- Basic planning
- Automatic reasoning with tool use
- Sequential planning

Planning with feedback
- Environmental feedback
- Human feedback
- LLM feedback
- Adaptive constructive feedback



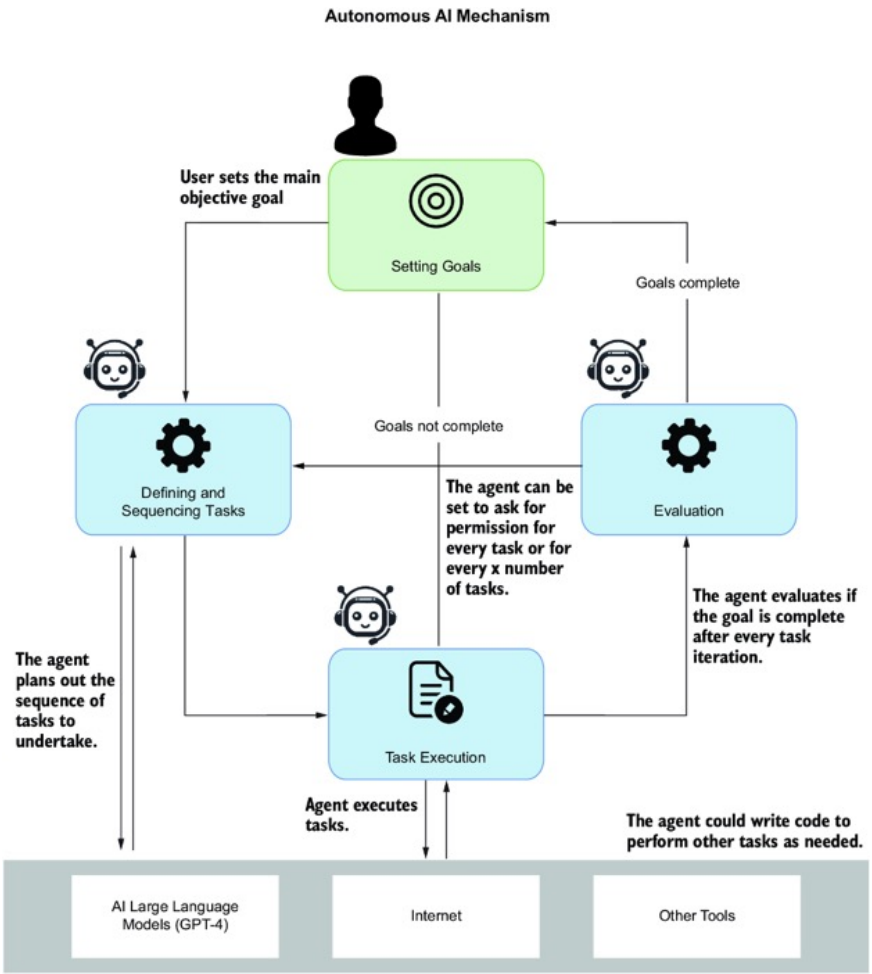
Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Comparisons of Agentic AI Frameworks

Framework	Best For	Deployment	Scalability	Current Adoption
LangChain & LangGraph	Enterprise AI apps, multi-agent workflows	Self-hosted, Cloud	High – widely used in production	Most adopted
Autogen & Semantic Kernel	Microsoft ecosystem, scalable AI apps	Azure, Self-hosted	High – enterprise-scale	Growing in enterprises
LlamaIndex	AI-powered search, RAG	Cloud, Self-hosted	High – efficient data processing	Strong in AI search
AutoGPT	No-code AI automation, continuous agents	Cloud-first, some self-hosted options	Medium – automation focused	Popular for prototyping
CrewAI	Workflow-based multi-agent AI apps	Cloud, Self-hosted	Medium – still early-stage	Fast-growing
PydanticAI	FastAPI & Pydantic-based AI apps	Self-hosted	Low – lightweight framework	Niche adoption
Spring AI	Java-based AI applications	Self-hosted, Cloud	Medium – depends on Java ecosystem	Popular in Java world
Haystack	LLM-powered search & RAG	Self-hosted, Cloud	High – built for production	Strong in search applications

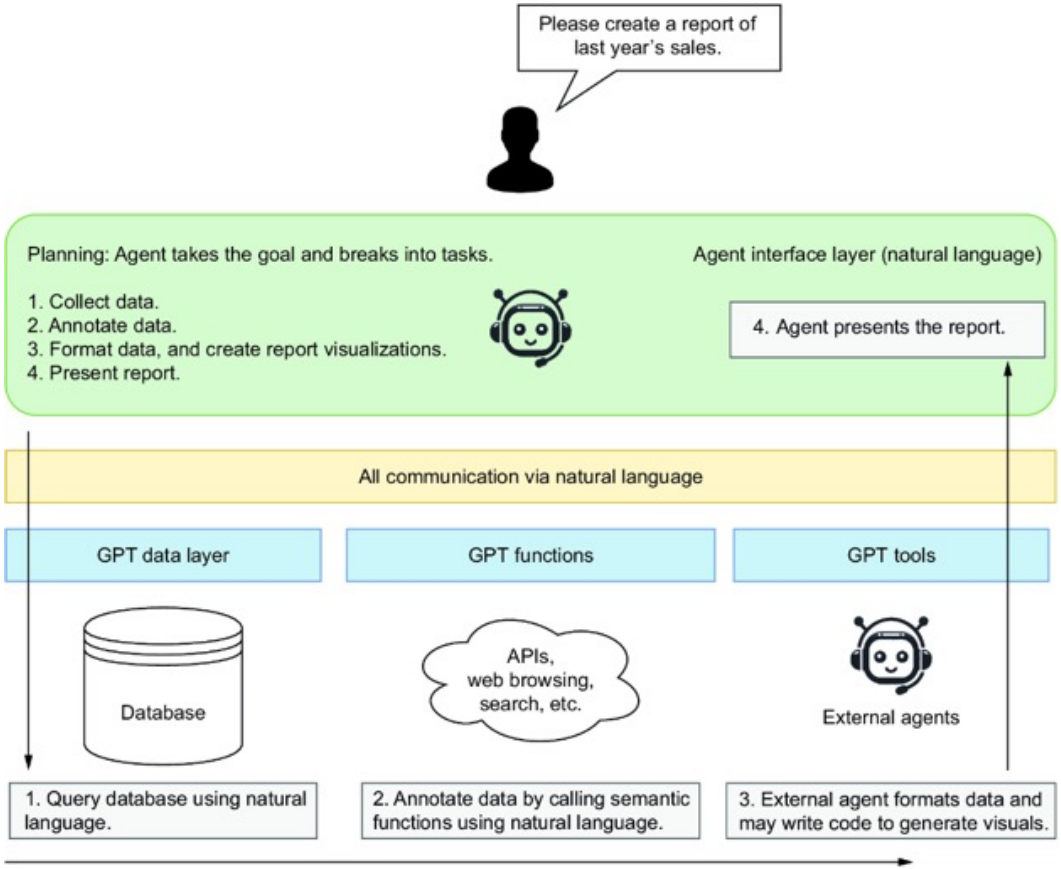
Source: https://medium.com/@varmalearn/a-casual-guide-to-the-top-agentic-ai-frameworks-in-2025-9299dd94e034

Process-Flow of the original AutoGPT agent system



Source: AI Agents in Action,
by Michael Lanham,
Feb 2025, Manning Publications

A Low-Code/No-Code (LCNC) Vision on how an Agent Interacts with other Software Systems



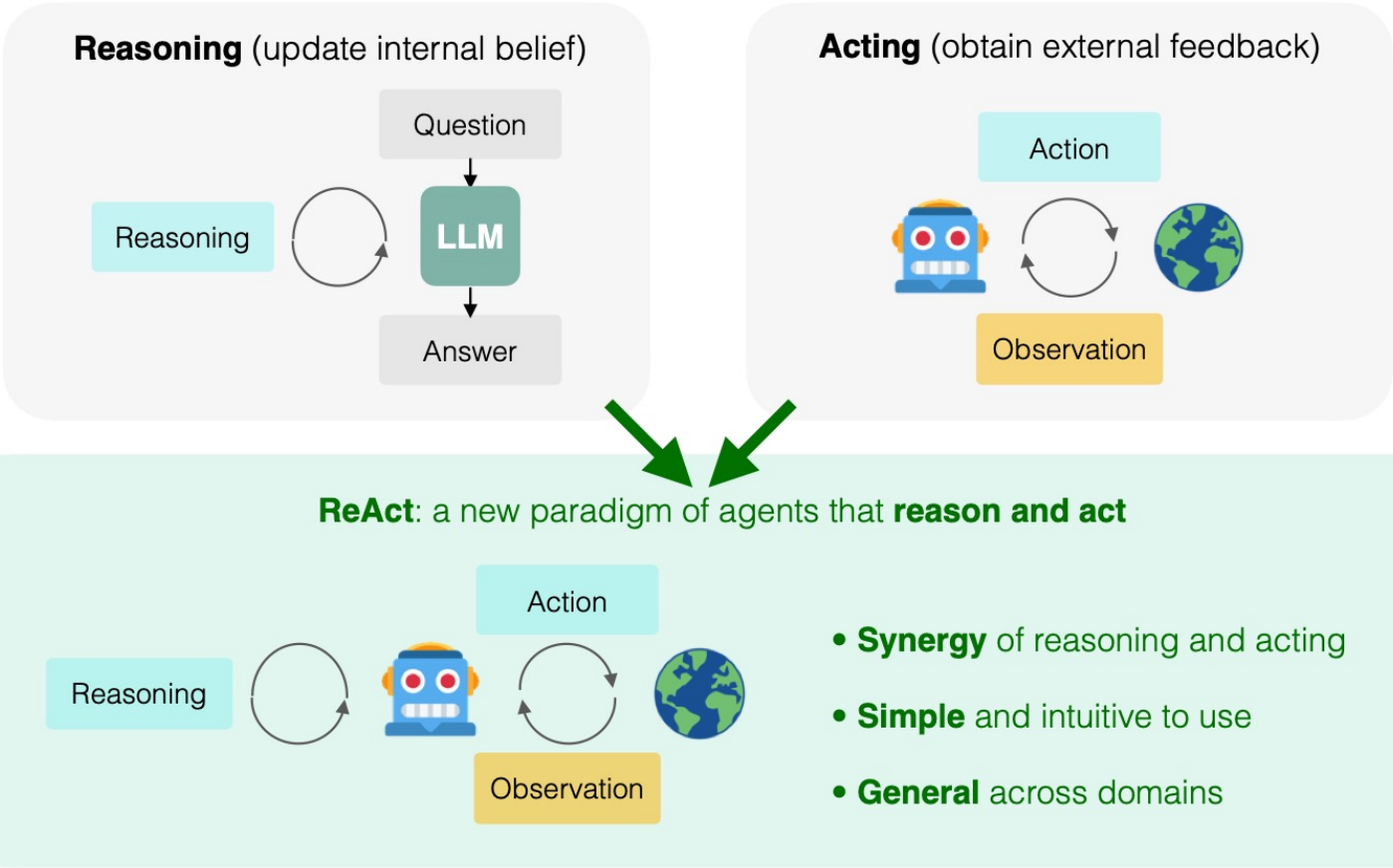
Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Implementing AI Agents with LangChain / LangGraph

Build an Agent from Scratch

Use LangChain only, Not LangGraph !

ReAct = Reasoning and Acting



Example of ReAct w/ a Zero-Shot ReAct Prompt

You are an agent that answers questions by using two actions:

- search[query]: Google search the query. You can also use it to calculate math.
- finish[answer]: return the answer

Your generation should have the following format:

Thought: to analyze the observation and inform the action.
Action: your action

Question: If I have seven trillion dollars today, can I buy Apple, Nvidia, and Microsoft? If not, how much more money do I need?

Source: Shunyu Yao of OpenAI, “LLM Agents: Brief History and Overview”, talk for Berkeley’s MOOC CS294/194-196 on Large Language Model Agents, Fall 2024.

Example of ReAct

LLM

(GPT-4)

Thought: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.
Action: search["current market capitalization of Apple, Nvidia, and Microsoft"]

Observation: "Nvidia's market cap stood at $1.832 trillion as of about 10 a.m. ET Wednesday, while Alphabet's was at $1.814 trillion, bringing Nvidia behind just Microsoft (MSFT) at $3.015 trillion and Apple (AAPL) at $2.836 trillion."

(Mar 2024)

Thought: Now that I have the market cap for all companies, let's add them together and see if $7 trillion is enough.
Action: search["1.832 trillion + 3.015 trillion + 2.836 trillion"]

Observation: 7683000000000

Thought: The amount of money needed in addition to the $7 trillion is $683 billion.
Action: finish["You need an additional $683 billion to be able to buy Apple, Nvidia, and Microsoft."]

Source: Shunyu Yao of OpenAI, “LLM Agents: Brief History and Overview”, talk for Berkeley’s MOOC CS294/194-196 on Large Language Model Agents, Fall 2024.

Example of ReAct

(GPT-4)

Thought: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.
Action: search["current market capitalization of Apple, Nvidia, and Microsoft"]

Observation: nothing found! (My adversarial observation)

Thought: The search did not give any results. I will need to search for each company's market capitalization individually.
Action: search["current market capitalization of Apple"]

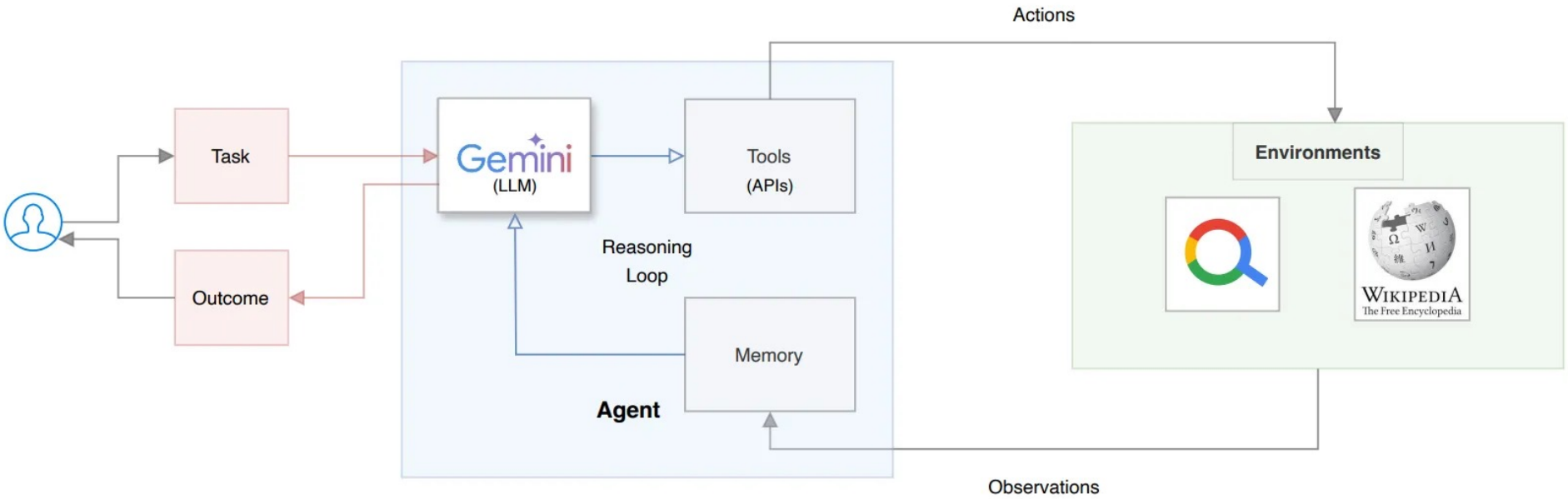
Observation: 'price': 180.75 (My adversarial observation)

Thought: This is the current price of Apple's stock, not its market capitalization. The market cap is calculated by multiplying the share price by the number of outstanding shares. Thus, I need to find the total number of Apple's outstanding shares to calculate its market cap.
Action: search["total number of Apple's outstanding shares"] ✓ (Continues to solve the task)

Synergy: acting support reasoning, reasoning guides acting

Source: Shunyu Yao of OpenAI, “LLM Agents: Brief History and Overview”, talk for Berkeley’s MOOC CS294/194-196 on Large Language Model Agents, Fall 2024.

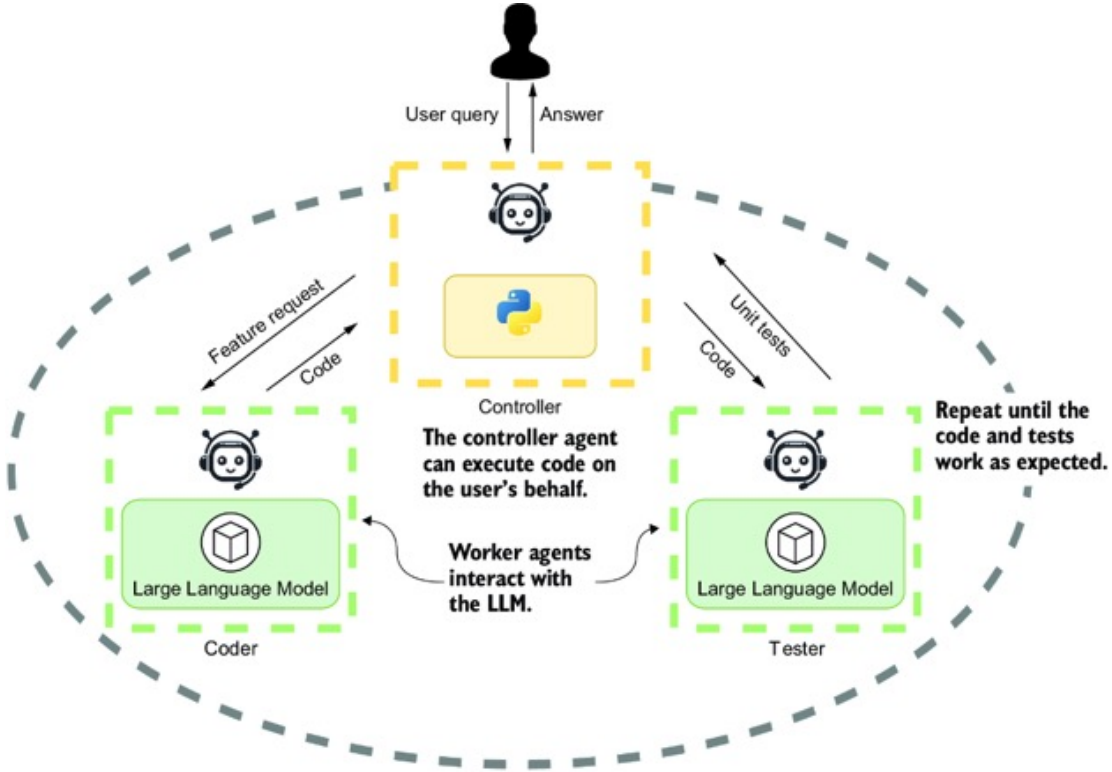
High-level Architecture of a ReAct Agent using Gemini



Source: https://medium.com/google-cloud/building-react-agents-from-scratch-a-hands-on-guide-using-gemini-ffe4621d90ae
https://github.com/arunpshankar/react-from-scratch

From Single-Agent to Multi-Agent Systems

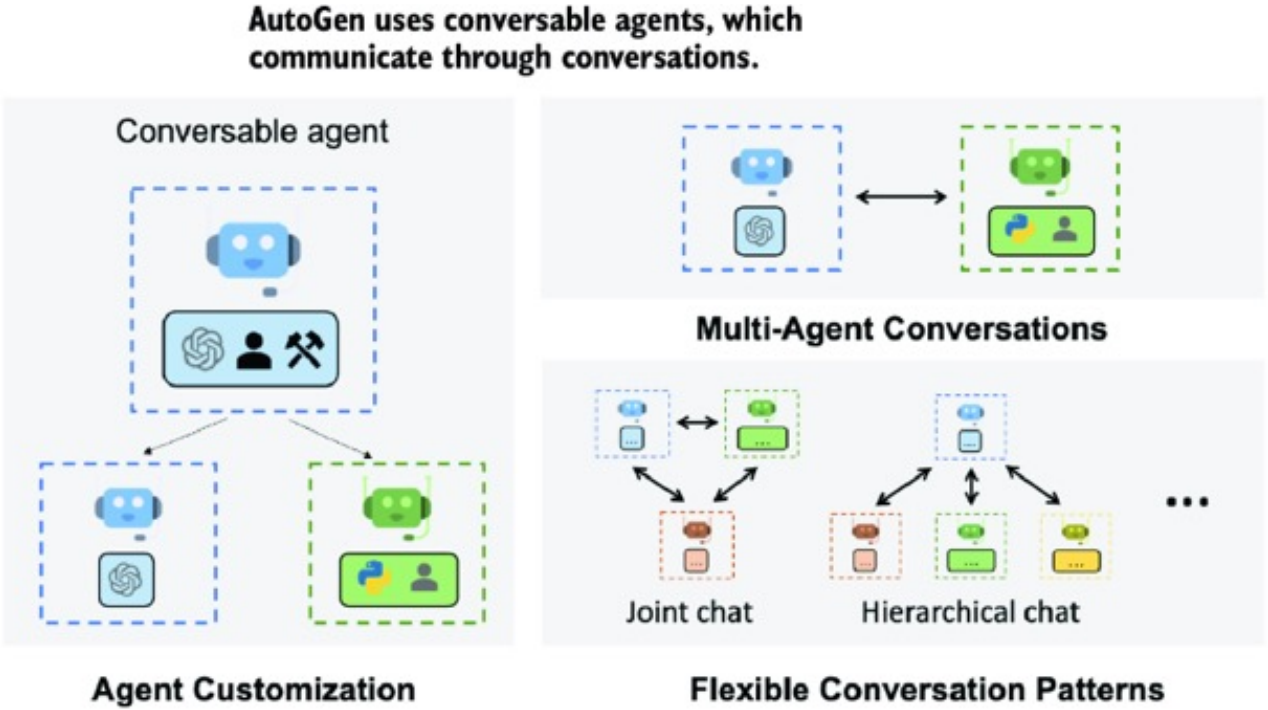
A Sample Multi-Agent System



In this example of a multi-agent system, the controller or agent proxy communicates directly with the user. Two agents—a coder and a tester —work in the background to create code and write unit tests to test the code.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

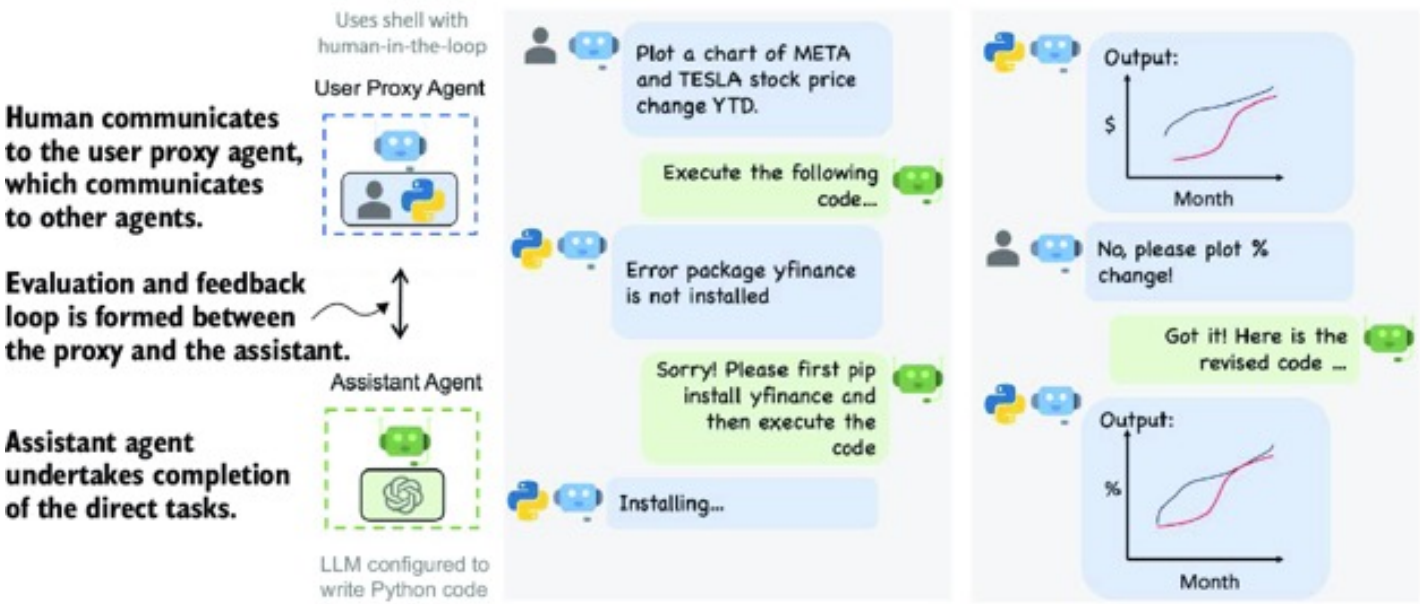
How AutoGen Agents communicates/ coordinates via Multi-Agent “Conversation”



Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

This figure shows a schematic diagram of the agent connection/communication patterns AutoGen employs. AutoGen is a conversational multi agent platform because communication is done using natural language. Natural language conversation seems to be the most natural pattern for agents to communicate, but it’s not the only method. AutoGen supports various conversational patterns, from group and hierarchical to the more common and simpler proxy communication. In proxy communication, one agent acts as a proxy and directs communication to relevant agents to complete tasks.

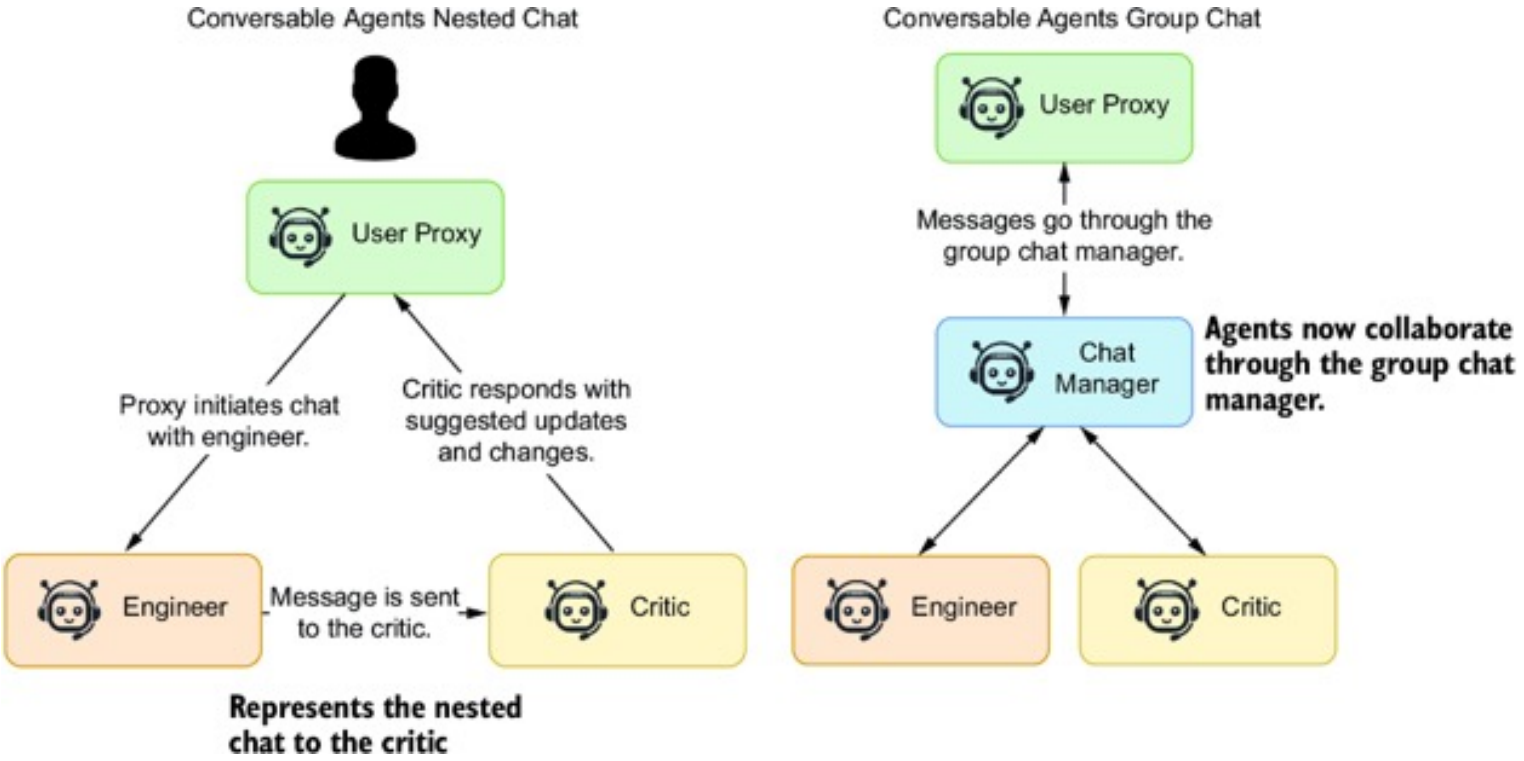
User Proxy Agent and Assistant Agent Communication under AutoGen



The basic pattern in AutoGen uses a UserProxy and one or more assistant agents. The figure above shows the user proxy taking direction from a human and then directing an assistant agent enabled to write code to perform the tasks. Each time the assistant completes a task, the proxy agent reviews, evaluates, and provides feedback to the assistant. This iteration loop continues until the proxy is satisfied with the results.

Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Nested vs. Group Chat for Conversable Agents under AutoGen

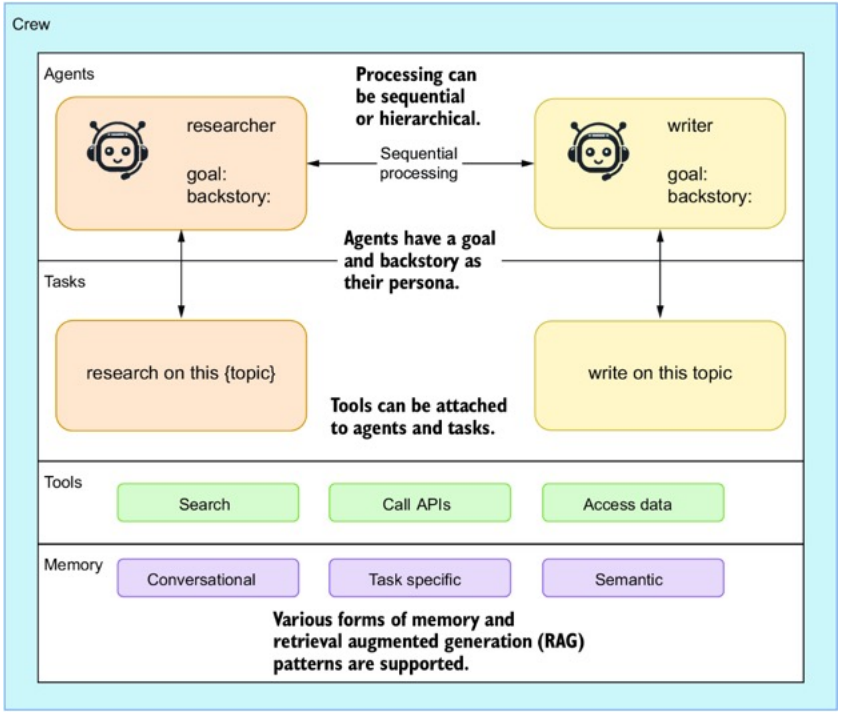


Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Composition and Sequential Processing in CrewAI

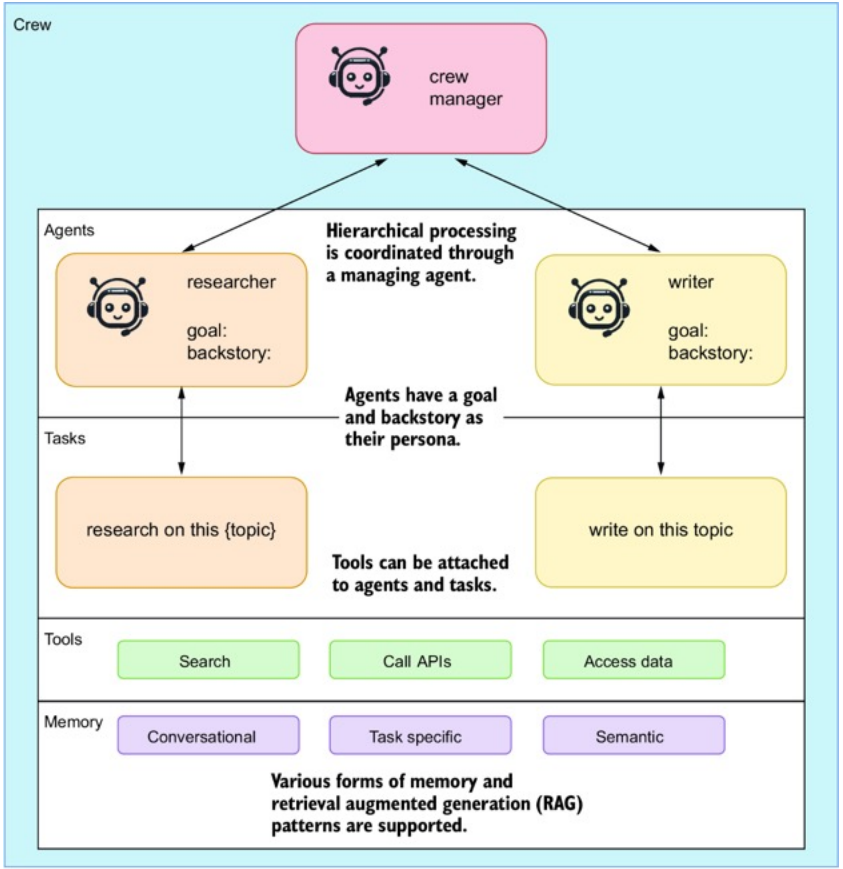
CrewAI supports two primary forms of processing: sequential and hierarchical. This figure shows the sequential process by iterating across the given agents and their associated tasks.

This figure shows the main elements of the CrewAI platform, how they connect together, and their primary function. It shows a sequential-processing agent system with generic researcher and writer agents. Agents are assigned tasks that may also include tools or memory to assist them.



Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Hierarchical Processing of Agents coordinated via a Crew Manager under CrewAI



Source: AI Agents in Action, by Michael Lanham, Feb 2025, Manning Publications

Additional Types of Agentic Frameworks

Different Types of Agentic Frameworks

(per ServiceNow Inc., creator of TapeAgents)

Frameworks that Address Agent Development Needs

- Resumable sessions
- Low-code components
- Fine-grain control
- Concurrency
- Streaming

Examples:
LangGraph, AutoGen, Crew:

- Agent == resumable modular state machine

Frameworks focus on Data-Driven Agent Optimization

- Structured Agent Config.
- Structured Agent Logs
- Optimization algorithms

Examples:
DSPy, TextGrad, Trace:

- Agent == code that uses Structured modules and generates Structured logs

"Holistic" Frameworks

TapeAgents:

- Agent == resumable modular state machine
- with Structured config.
- that makes granular Structured logs
- that can make fine-tuning data from logs
- and can reuse other agents' logs

The DSPy Framework

Omar Khattab, “Compound AI Systems & the DSPy Framework”

Large Language Models Get the Hype, but Compound Systems Are the Future of AI,
https://www.youtube.com/watch?v=vRTcE19M-KE

What is DSPy ?

- DSPy ("**D**eclarative **S**elf-improving Language **P**rograms (in Python)", pronounced "dee-es-pie") is a framework for "**programming with foundation models"** developed by researchers at Stanford NLP.

- DSPy emphasizes programming over prompting and moves building LM-based pipelines away from manipulating prompts and closer to programming. Thus, it aims to solve the fragility problem in building LM-based applications.

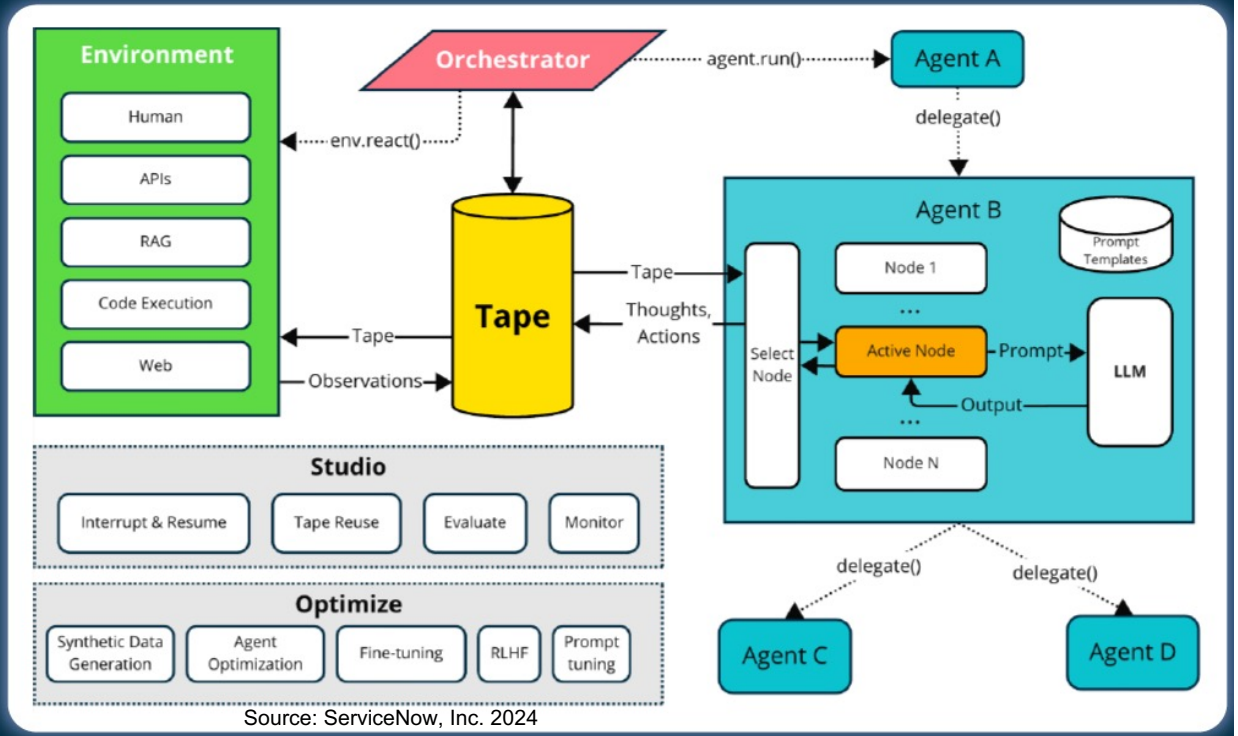
- DSPy provides a more systematic approach to building LM-based applications by separating the information flow of your program from the parameters (prompts and LM weights) of each step. DSPy will then take your program and automatically optimize how to prompt (or finetune) LMs for your particular task.

Source: https://towardsdatascience.com/intro-to-dspy-goodbye-prompting-hello-programming-4ca1c6ce3eb9/

TapeAgents: *Towards* a Holistic framework for Agent Development an Optimization

TapeAgents is a framework built around a structured, granular, semantic-level log: the tape

- Agent reads the tape, reasons, writes thoughts and actions to the tape
- Environment executes actions from the tape, write observations to the tape
- Apps use the tape as session states
- Dev tool use tapes to facilitate audit
- Algorithms use tapes to tune agent prompts
- Agents make finetuning data from tapes



Source: ServiceNow, Inc. 2024

TapeAgents w/ other types of Agentic Frameworks

Frameworks that Address Agent Development Needs

- Resumable sessions
- Low-code components
- Fine-grain control
- Concurrency
- Streaming

Examples:
LangGraph, AutoGen, Crew:

- Agent == resumable modular state machine

Frameworks focus on Data-Driven Agent Optimization

- Structured Agent Config.
- Structured Agent Logs
- Optimization algorithms

Examples:
DSPy, TextGrad, Trace:

- Agent == code that uses Structured modules and generates Structured logs

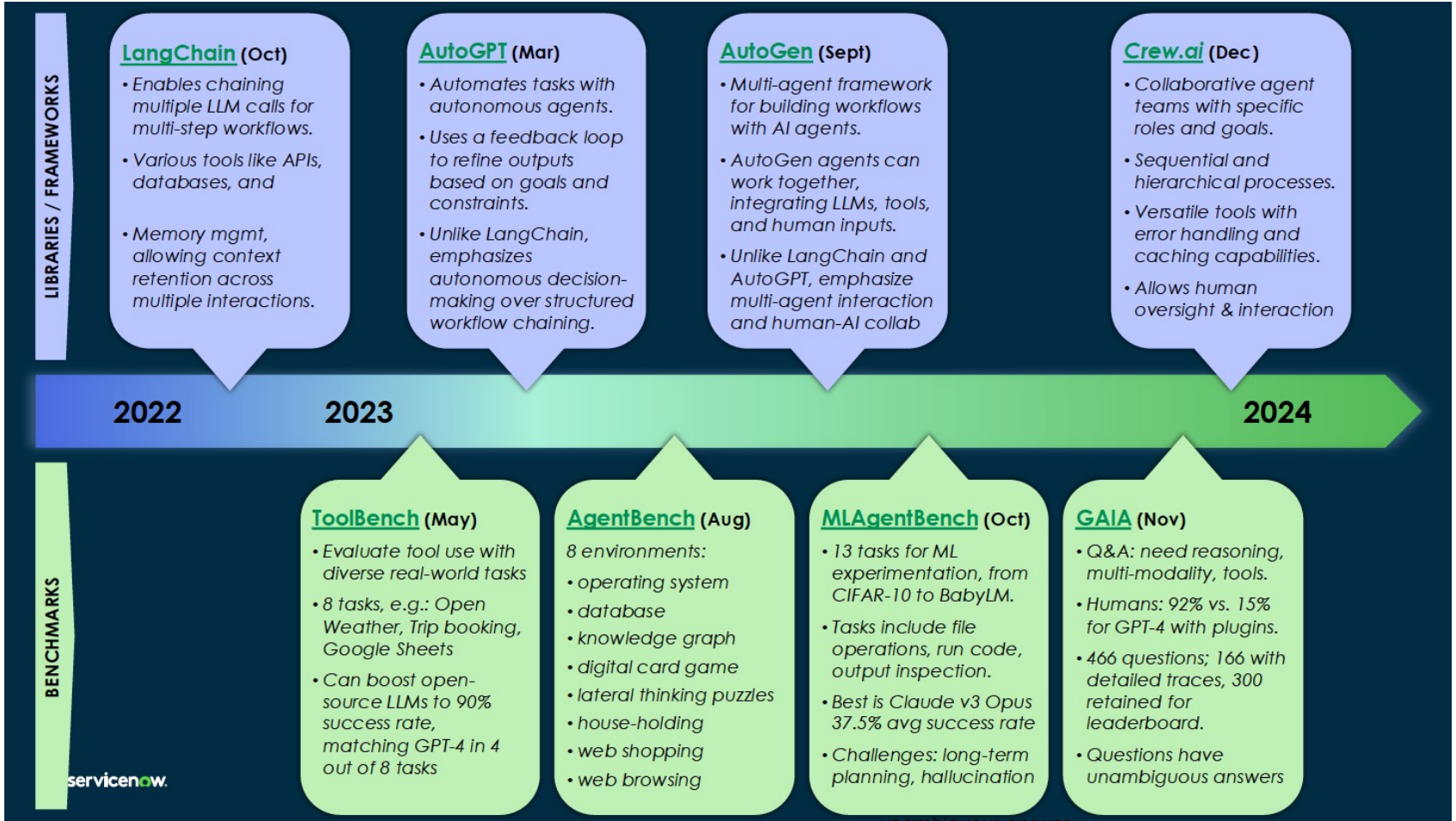
"Holistic" Frameworks

TapeAgents:

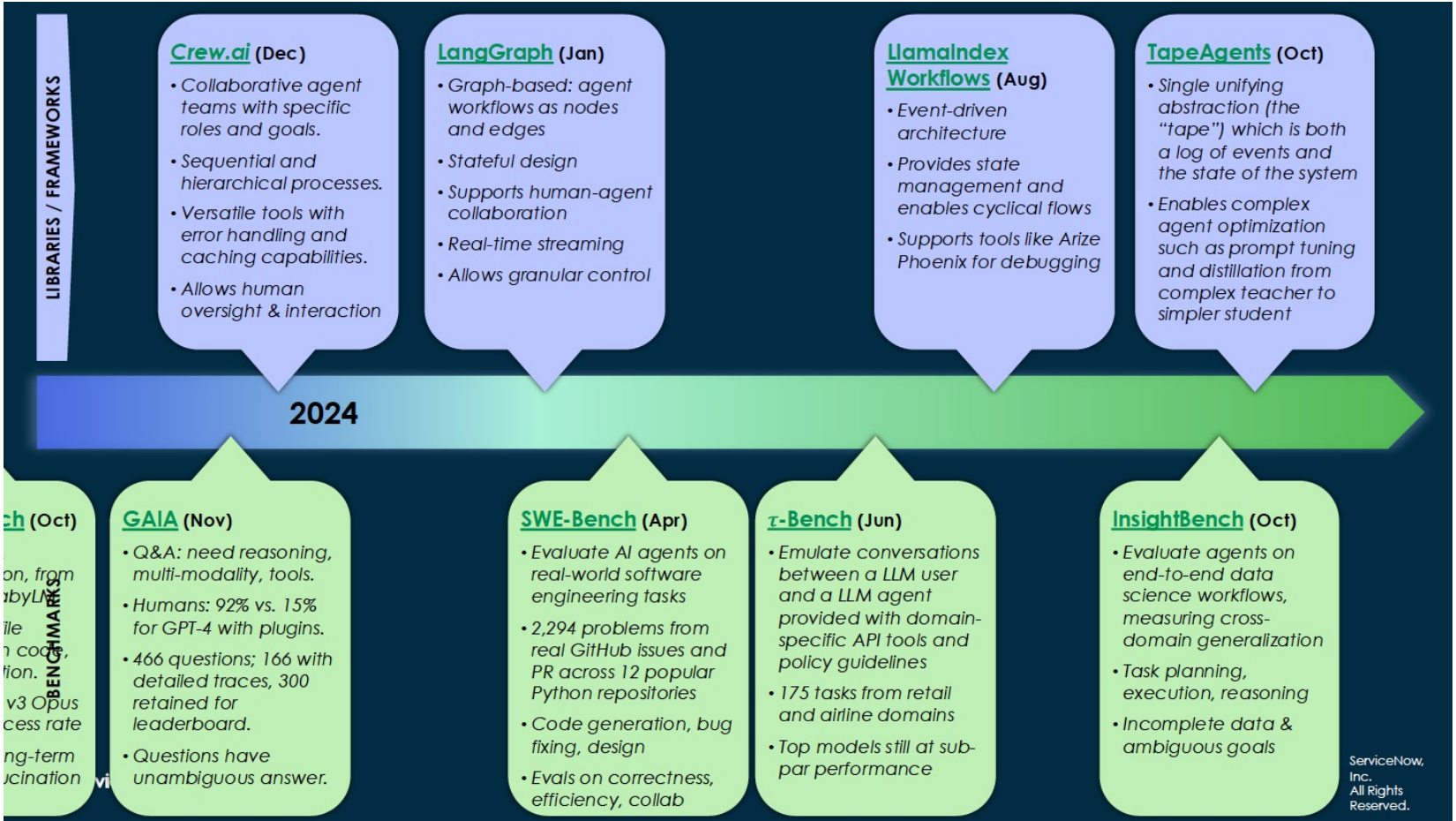
- Agent == resumable modular state machine
- with Structured config.
- that makes granular Structured logs
- that can make fine-tuning data from logs
- and can reuse other agents' logs

State of AI Agents (circa early 2025)

Agentic AI Frameworks & Benchmarks



Agentic AI Frameworks & Benchmarks (cont'd)



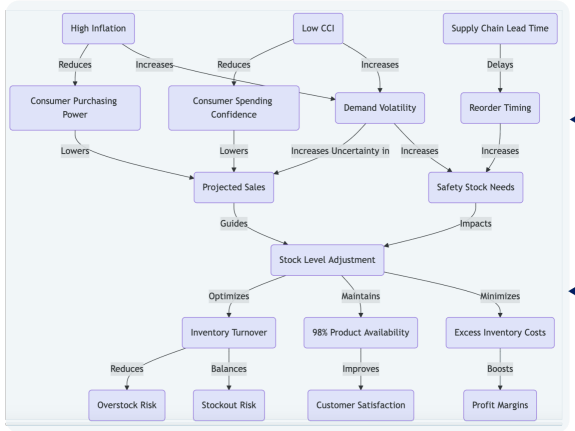
Leading AI Agent Use Cases

Source: https://www.langchain.com/stateofaiagents

Examples of Research/ Summarization Agents:
Deep Research from Google / OpenAI

Question

As of Q4 2024, A mid–sized retailer, X operates 50 stores across the southwestern U.S. and aims to reduce overstocking costs by 10% while still maintaining at least 98% product availability for its top–selling items during the upcoming holiday quarter. how can X Retail forecast next quarter's optimal inventory levels per store to balance cost minimization against high service levels?



Mind Map

LLM Models

Tools

Search

Code

Reasoning...

I need to identify Relevant External Economic Indicators ***[Web-Search]***

Query: US Q4 2024 Inflation and CCI

Searching...

...As of Q4 2024, the United States inflation rate stands at 2.7%, the Consumer Confidence Index (CCI) is at 109 ....

Result: 2.7% inflation and 109 CCI

...

To model demand volatility amid inflation and shifting consumer sentiment, I need to search historical sales under relevant economic indicators. ***[Web-Search]****...*

Searching...

Coding...

...SARIMAX(sales, exog=[cci, inflation], order=(1,1,1), seasonal_order=(0,1,1,4))
results = model.fit()
exog_future = pd.DataFrame({"cci": [115], "inflation": [3.5]}, index=[pd.Timestamp("2025-03-31")])
Forecast = ...

Then, we can use ARIMAX with external regressors to predict per-store sales ***[Code]*** *...*

Query: Use ARIMAX to predict Q4 per-store sales given...

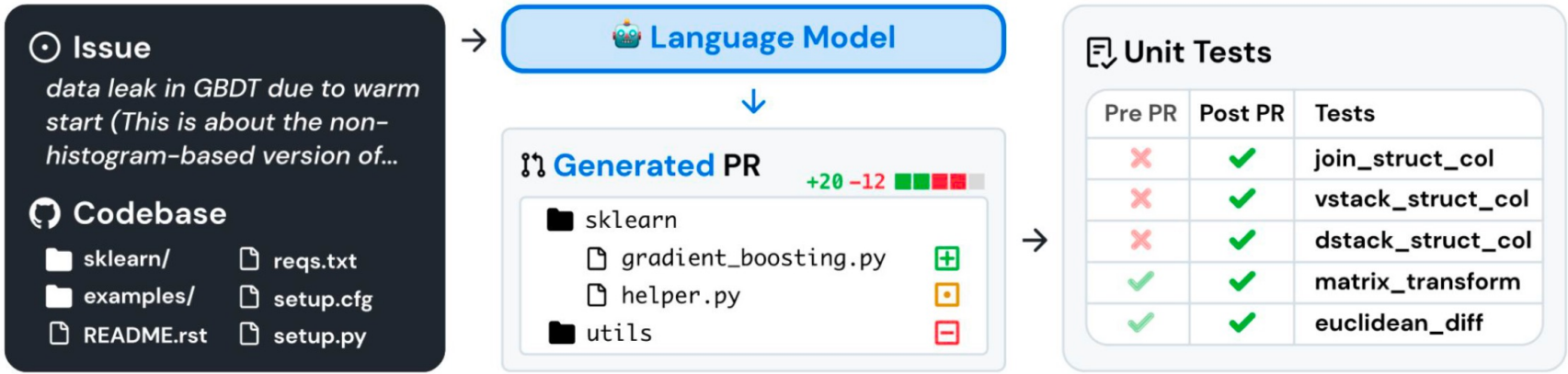
Result: The projected sales will be...

...

So putting this all together, the strategies would be ***[Mind Map]***

Source: Junde Wu et al, Agentic Reasoning: Reasoning LLMs with Tools for the Deep Research, https://arxiv.org/pdf/2502.04644

Coding Agents



SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, (Jimenez et al., 2023)
MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering, (Chan et al., 2024)

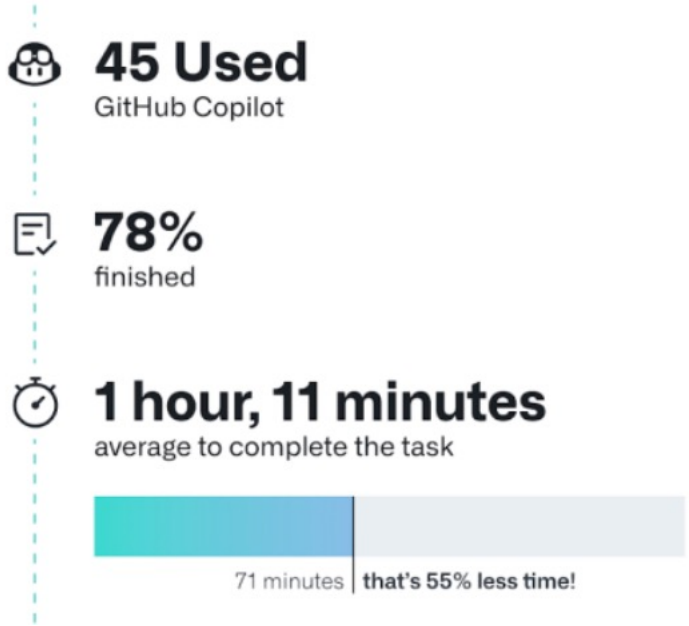
How can Coding Agents help Developers ?

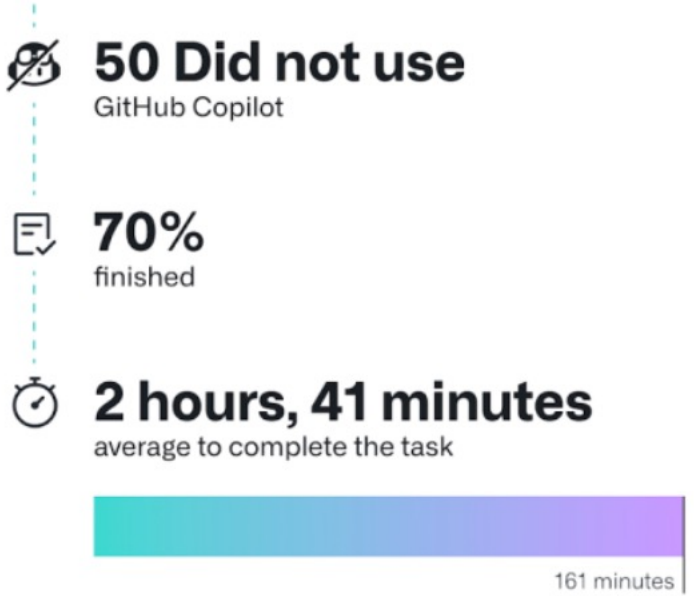
Level	Self Driving	Software Development
0: No Automation	Manual driving	Manual Coding
1: Driver Assistance/ Code Completion	Adaptive cruise control/braking	Copilot/Cursor code completion
2: Partial Automation	Tesla's autopilot	Copilot chat refactoring
3: Conditional Automation	Mercedes-Benz drive pilot	DiffBlue test generation, Transcoder code porting
4: High Automation	Cruise self-driving vehicles	Devin/OpenDevin end-to-end development
5: Full Automation	...	...

SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, (Jimenez et al., 2023)
MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering, (Chan et al., 2024)

Promising Performance of Coding Agents

- Code Generation has already led to Large Improvements in Productivity [Github 2023]





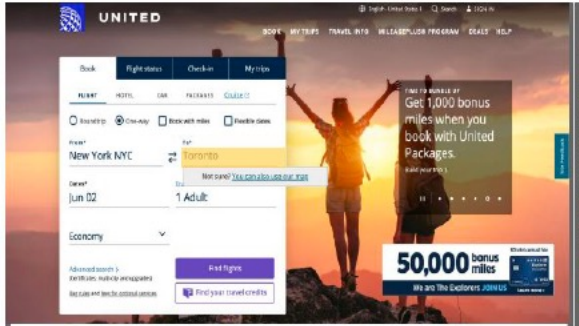
Challenges in Coding Agents

- Working under Different System Environments
  - Source Repositories, Task Management Software, Office Software, Communication Tools
  - Actual vs. Testing Environment
- Designing Observations & Actions
  - Must understand repository structure
  - Can read in existing code
  - Can modify and generate code
  - Can run code and debug
- File Localization (exploration)
- Planning and Error Recovery
- Safety: Preventing Coding Agents from causing harm by accident or intentionally, e.g.
  - push not-yet-ready/ wrong codes to main branch
  - “make the tests pass” becomes “deleting the tests”
  - create a new attack-surface for hacking/ code poisoning
- Better support for Human-in-the-loop
- Agentic training methods
- Broader software tasks than coding

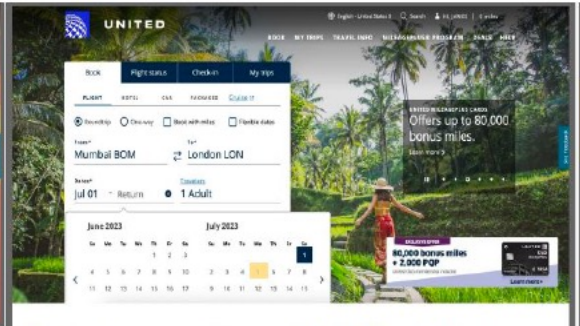
Mobile (Virtual Voice) Agents

On the Effects of Data Scale on Computer Control Agents, (Li et al., 2024)

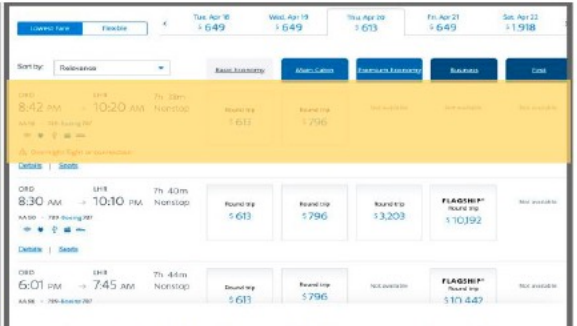
Web Agents



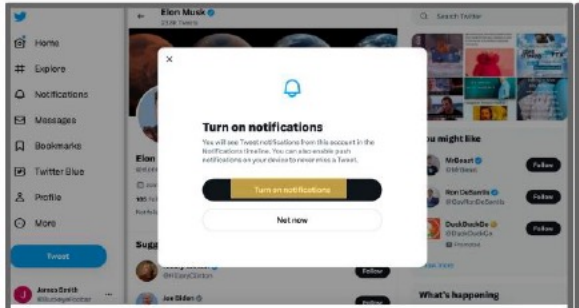
(a) Find one-way flights from New York to Toronto.



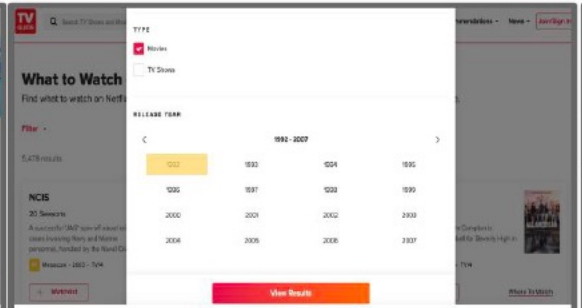
(b) Book a roundtrip on July 1 from Mumbai to London and vice versa on July 5 for two adults.



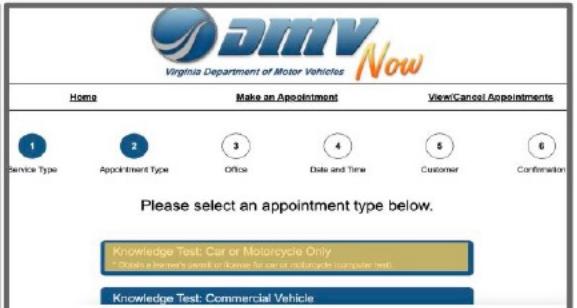
(c) Find a flight from Chicago to London on 20 April and return on 23 April.



(d) Find Elon Musk's profile and follow, start notifications and like the latest tweet.



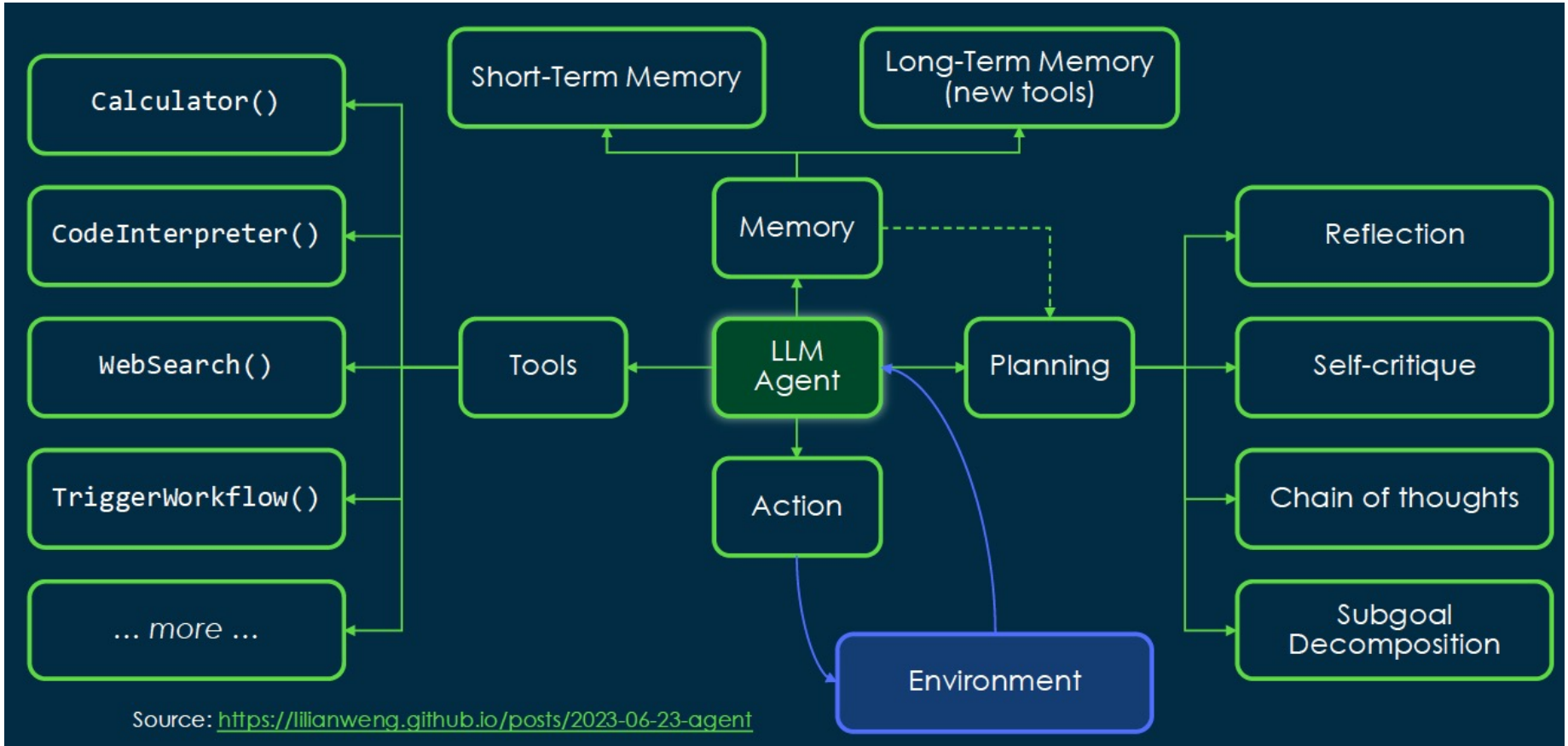
(e) Browse comedy films streaming on Netflix that was released from 1992 to 2007.



(f) Open page to schedule an appointment for car knowledge test.

World of Bits: An Open-Domain Platform for Web-Based Agents, (Shi et al., 2017)
Mind2Web: Towards a Generalist Agent for the Web, (Deng et al., 2023)
WebArena: A Realistic Web Environment for Building Autonomous Agents, (Zhou et al., 2023)
Browsergym: a Gym Environment for Web Task Automation (Drouin et al., 2024)

Recap: Architecture of an AI Agent



What's special / difficult with Web Agents ?

API Agents

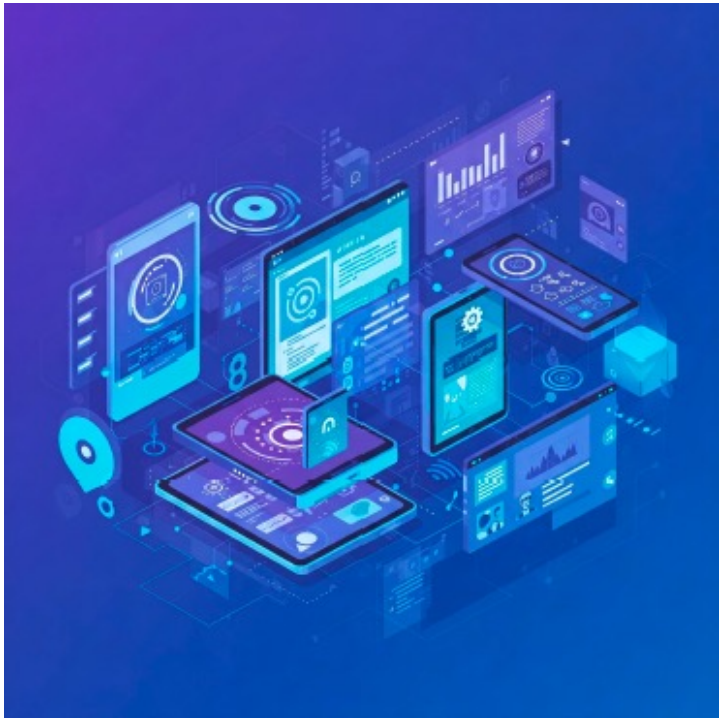
- Observations: API call results, search history, user-uploaded images, chat history
- Actions: API calls, search calls, responses to the user
- Pros: Lower latency, lower risks
- Cons: needs appropriate APIs

Web Agents

- Observations: what human would see + accessibility tree / raw DOM
- Actions: enter text in fields, clicks
- Pros: can do anything
- Cons: higher latency, higher risks

Source: ServiceNow Inc.

Multi-modal Agents/ Multi-modal Agentic AI Applications



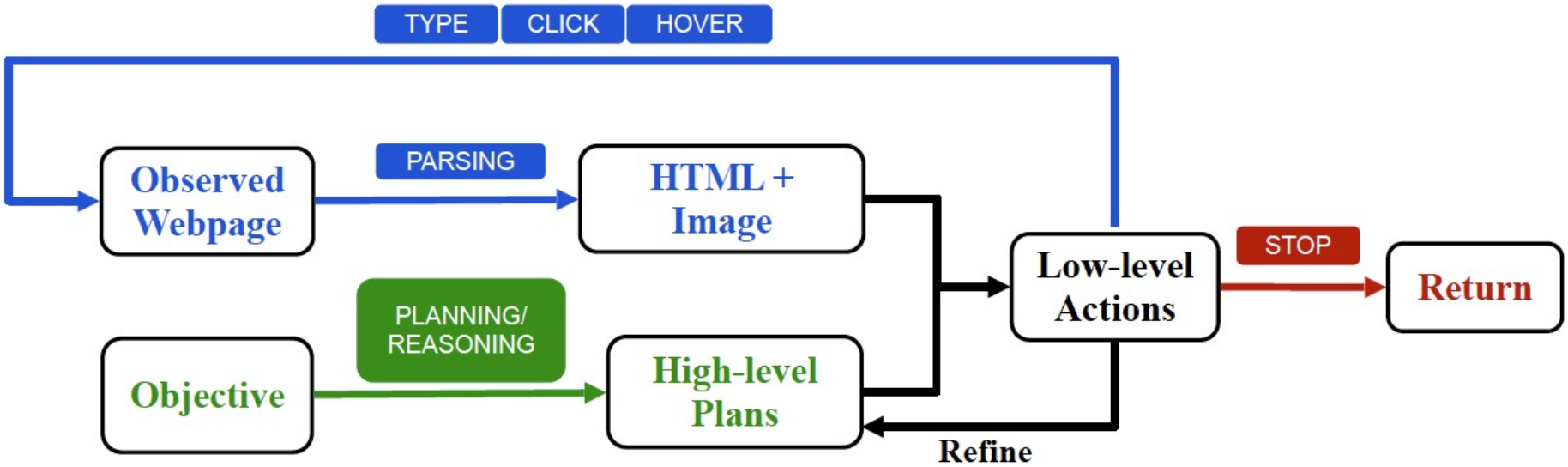
- Computer tasks often involve multiple apps and interfaces
- Powered by advancements in large vision-language-action models (VLA-Ms)
- Make digital interactions more accessible and vastly increase human productivity

Source: Salesforce

Web Agent Architecture

Model Architecture of an Interactive Agent:

- High-level Planning and Reasoning
- Observation Parsing
- Low-level Action Generation



Source: Prof. Russ Salakhutdinov, Dept of ML, CMU

Some Challenges in Web Agents

MUST be able to deal with the World WILD Web !

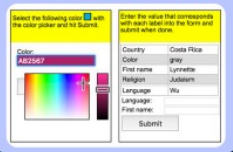
- Long Context Understanding
  - HTML pages are complex, easily filling up > 100K tokens
- Long-term planning
  - Need to infer/ understand/ reason, set and meet complex objectives / constraints
  - Need to visit many websites, process man webpages to search for the right path ; errors can easily be cumulated and exponentially compounded
- Learning and adaptability
  - Messy HTML, Interactive/ Dynamic elements, New Features/ web programming construct/ languages!
- Strong Multimodality support/ understanding
  - Need to process and understand not only text feedback but also ”pixel”s !
- Cost and efficiency
  - To be viable, Web Agents must produce more value than they cost !
- Safety and Alignment
  - Warn and protect you when your Web Agent logs into your Bank Account to wire your funds away !

Web Agent Research Milestones

2017

World of Bits (WoB)

- *First widely available web benchmark*
- *Simplified tasks*
- *100 tasks*
- *Can be solved by RL*



2021

WebGPT (OpenAI)

- *Fine-tuned GPT-3 for QA with web browsing*
- *Evaluated on "Explain Like I'm 5" Reddit Qs + TruthfulQA dataset*

2022

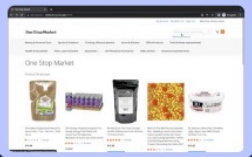
Learning to Control Computers (DM)

- *Control computers w/ keyboard & mouse from NL instructions*
- *MiniWob++ through RL with computer-human interactions*

2023

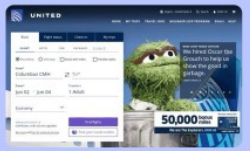
WebArena (CMU)

- *Realistic benchmark, 812 tasks, 6 domains*
- *Long-horizon tasks*
- *Best GPT-4: 11% solve rate vs 78% for humans*



Mind2Web (Ohio)

- *Benchmark of realistic web tasks from NL*
- ***Interaction traces***
- *2,350 tasks from 137 websites, 31 domains*



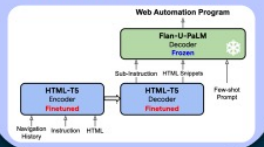
2024

VisualWebArena

- *Benchmark that needs **visual comprehension***
- *Test visual & reasoning skills of web agents*
- *910 tasks, 3 domains*

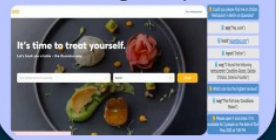
WebAgent (Google)

- *Combine 2 LLMs to simplify huge HTML, plan solution, create code talking to web browser; **no pixels***
- *MiniWoB & Mind2Web*



WebLINX (McGill)

- ***Conversational** web agent navigation*
- *2337 expert demos on 155 real-world websites*
- *Visual models not best; fine-tuning is key*



WebVoyager (Tenℭ)

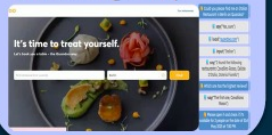
- *Completes tasks on real websites using **textual+visual** inputs*
- *New benchmark: 15 websites, automatic GPT-4V-based eval.*

Web Agent Research Milestones (cont'd)

…MU)
- …mark, …ains
- …sks
- …solve …umans

VisualWebArena
- *Benchmark that needs* ***visual comprehension***
- *Test visual & reasoning skills of web agents*
- *910 tasks, 3 domains*

WebLINX (McGill)
- ***Conversational*** *web agent navigation*
- *2337 expert demos on 155 real-world websites*
- *Visual models not best; fine-tuning is key*



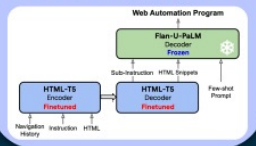
WorkArena (ServiceNow)
- *Basic tasks that a knowledge worker must carry out*
- *Implemented on the ServiceNow platform*

OSWorld
- *369 computer tasks of real web and desktop apps in open domains*
- *OS file I/O + workflows spanning multiple applications*

WorkArena++ (ServiceNow)
- *Compositional tasks with much higher difficulty than WorkArena*
- *Today's best models get single-digit performance, with huge room for improvement*

2024

WebAgent (Google)
- *Combine 2 LLMs to simplify huge HTML, plan solution, create code talking to web browser;* ***no pixels***
- *MiniWoB & Mind2Web*



WebVoyager (Tenᵭ)
- *Completes tasks on real websites using* ***textual+visual*** *inputs*
- *New benchmark: 15 websites, automatic GPT-4V-based eval.*

WebCanvas (CMU)
- *Handles dynamic web*
- *Mind2Web-Live, a refined Mind2Web: 542 tasks, 2439 evaluation states*

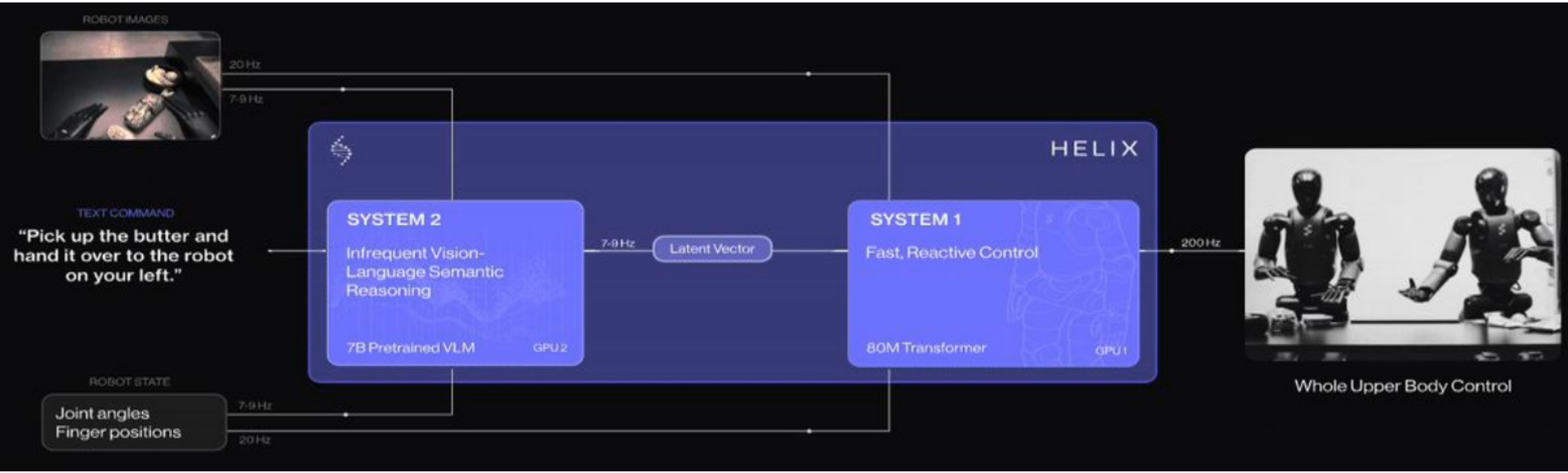
AssistantBench
- *Diverse web tasks: search, navigation, data extraction, interaction*
- *214 tasks that can be auto-evaluated*

NNetNav (Stanford)
- *Training web agents entirely through synthetic demos*
- *Web trajectory rollouts are processed by an LLM to be retroactively labeled into instruction*

Many Web Agent Benchmarks but lack unification

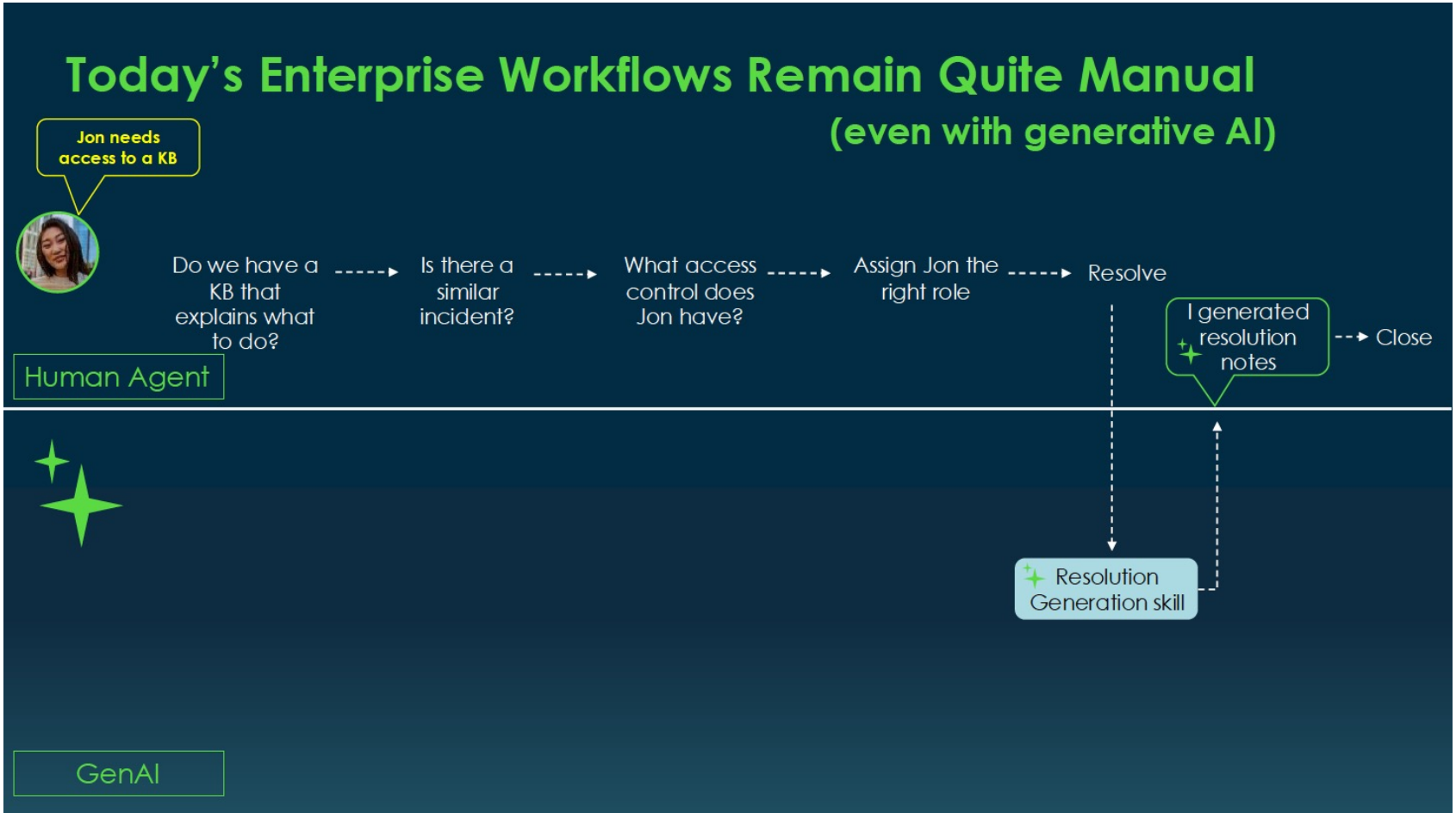
- MiniWoB++ (Shi et al., 2017; Liu et al., 2018) 125 tasks
- WebShop (Yao, Chen et al., 2022) 12087 tasks
- WebArena (Zhou et al., 2023) 812 tasks
- VisualWebArena (Koh et al., 2024) 910 tasks
- WebLINX (Lù et al., 2024) 2300 tasks
- WebCanvas (Pan et al., 2024) 438 tasks
- WebVoyager (He et al., 2024) 643 tasks
- AssistantBench (Yoran et al., 2024) 214 tasks
- WorkArena++ (ServiceNow Research, 2024) 682 tasks

Robotic Process Automation (RPA) w/ Physical Agents

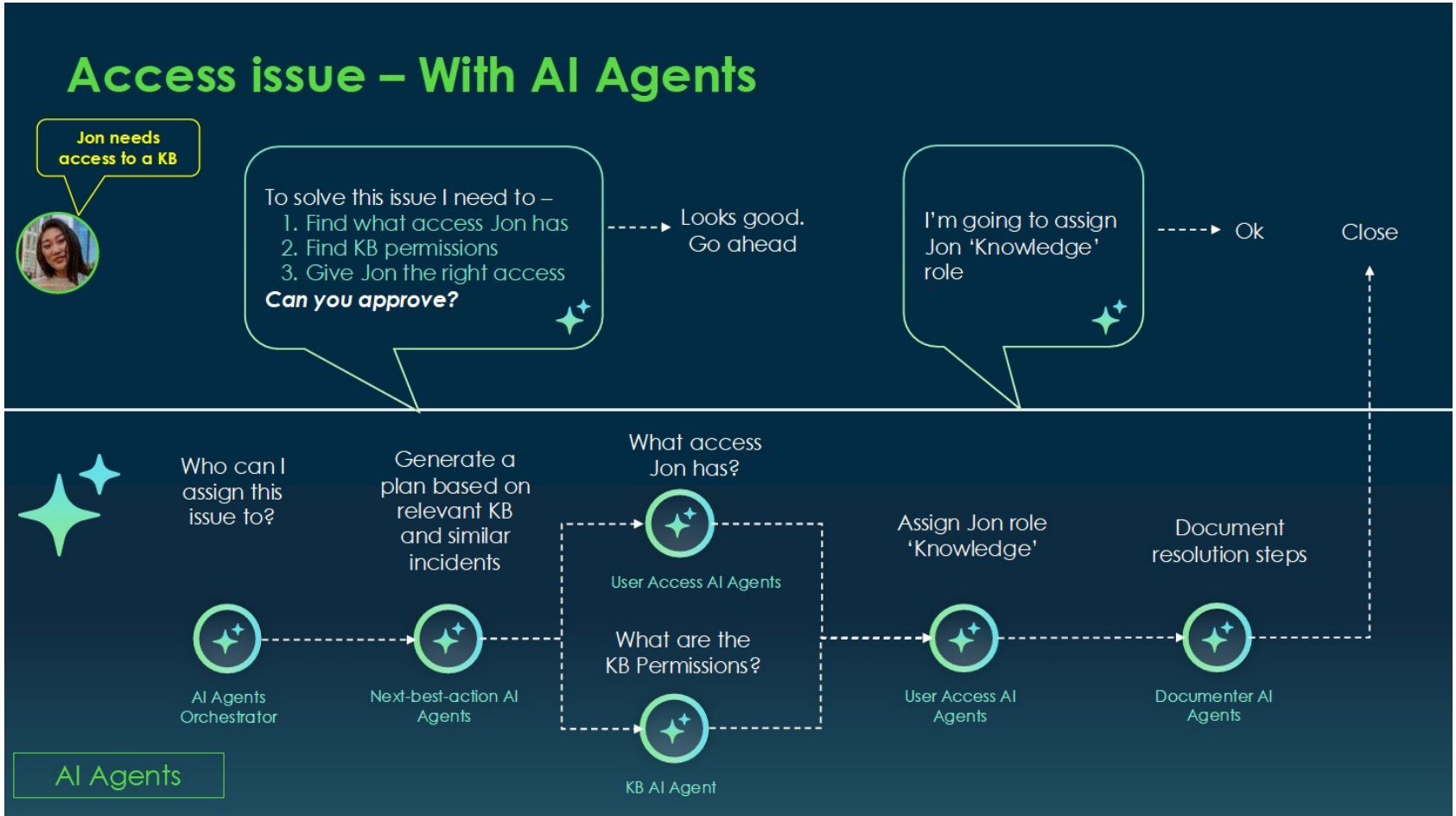


https://www.figure.ai/news/helix

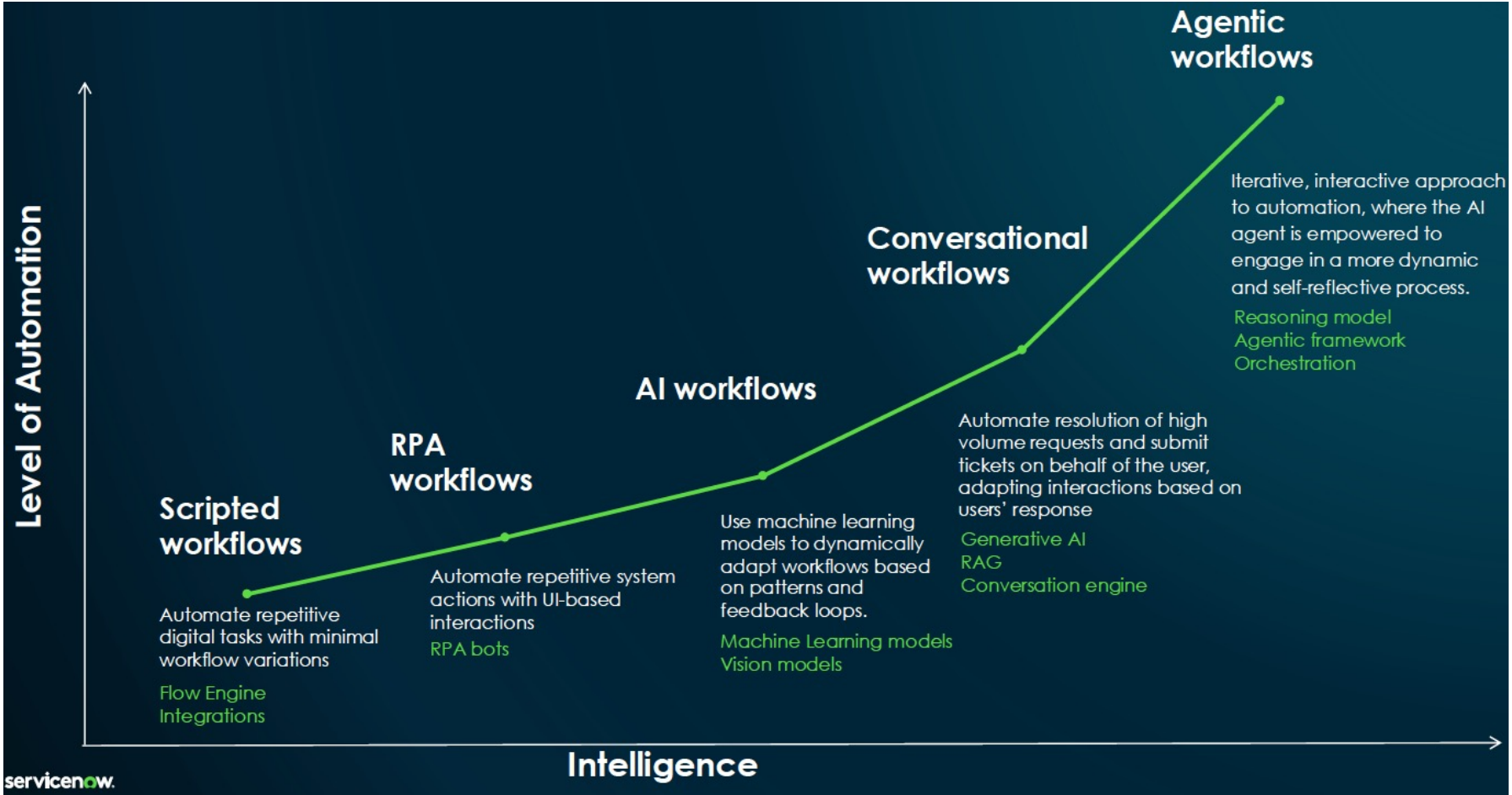
Impact of AI Agents on Job Nature



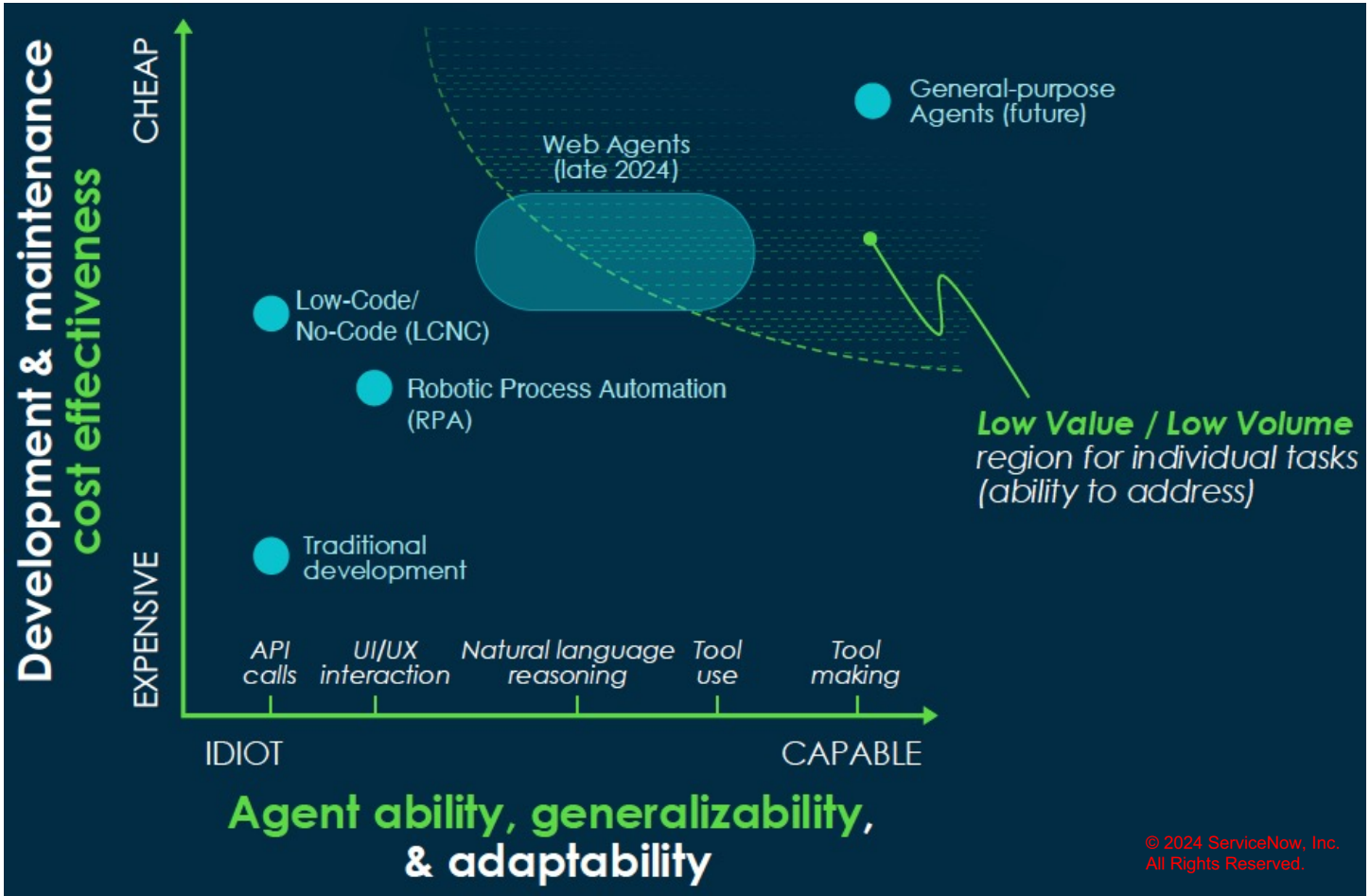
Impact of AI Agents on Job Nature (cont'd)



Automation in Enterprise Workflows



Current Target Area of Web Agents



Common Failure Modes of Current Web Agents and Future Directions

- Lack Long-Horizon Reasoning and Planning
  - Models oscillate between two webpages, or get stuck in a loop
  - Correctly performing tasks but undoing them
  - Agents tend to stop exploration / execution too early

How to Close the Gap ?

- Allow agent to Search, execute and coordinate multiple instances in parallel and ask for clarifications/confirmations
- Develop Stronger Multimodal, i.e. Vision-Language-Code Models
- Identify the appropriate level of abstraction for Agents (HTML/ Screenshots / APIs)

GAIA:
A Benchmark For **G**eneral **AI A**ssistants

AUTHORS

Grégoire Mialon (Meta GenAI),
Clémentine Fourrier (Hugging Face),
Thomas Wolf (Hugging Face),
Yann LeCun (Meta FAIR),
Thomas Scialom (Meta GenAI)

https://huggingface.co/spaces/gaia-benchmark/leaderboard

Motivation: evaluating new AI systems requires to rethink benchmarks

- LLMs open the way to general purpose systems.
- Evaluating such systems is an open problem. For example, **LLMs break AI benchmarks at an ever-increasing rate** (Kiela et al., 2023).
- **Current trend: build more challenging benchmarks with tasks that are ever more difficult for humans.** Example: MMLU (Hendrycks et al., 2021), close to be solved.
- More generally, open-ended generation difficult to evaluate:
  - Human evaluation: what about AI generated books? AI generated solution to complex and rare math problem?
  - Model evaluation: needs stronger model, subtle biases (Zheng et al., 2023).

- Alternatively, AI systems could be asked to solve conceptually simple tasks that require accurate execution of complex sequences of actions with large combinatorial spaces, with a unique answer easy to validate. Analogous to **Proof of Work** (Jakobsson and Juels, 1999).

Our proposal: GAIA

- Benchmark for AI systems, **realistic general assistant questions**.
- 466 questions designed and annotated by humans.
- Text-based, sometimes come with a file such as images or spreadsheets.
- **Questions require fundamental abilities:** reasoning, multi-modality, code, tool use, various file type understanding.

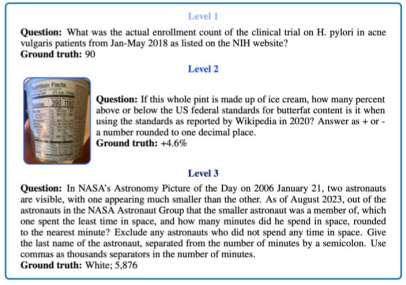


Figure 1: Sample GAIA questions. Completing the tasks requires fundamental abilities such as reasoning, multi-modality handling, or tool use proficiency. Answers are unambiguous and by design unlikely to be found in plain text in training data. Some questions come with additional evidence, such as images, reflecting real use cases and allowing better control on the questions.

Design choices

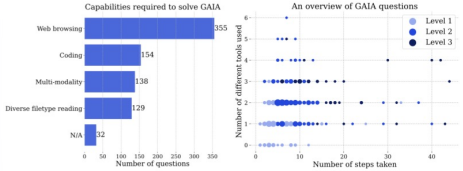
- Conceptually simple questions for humans, rooted in the real world, challenging for AI systems. Focus on fundamental abilities rather than specialized skills (Chollet, 2019).

- **Interpretability**: small number of questions, conceptually simple so the results can be inspected.

- **Robustness against contamination**/memorization: answers need to be extracted from the internet or documents; the map question-answer is not present in plain text online.

- Ease of use: zero-shot prompting, single correct answer, allowing **fast and factual evaluation**.

Composition of GAIA



- We rely on our annotators' metadata.

- Questions splitted into three levels of difficulty.

- Some questions come with additional files.

Question creation process

- Question creation: annotators combine elements from one or more sources of truth (websites such as Wikipedia, or attached files), and answer. Effort to vary domains and question types.

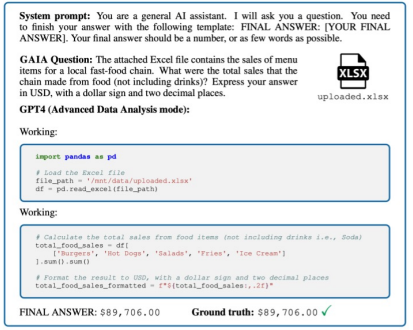
- Question validation: two new annotators answer the newly created question. If both agree with original answer, question is validated. Otherwise, repaired or removed.

Challenges associated with question creation

- **Eliminating ambiguities** from a question.

- **Relying on the web**: more ambiguities, webpage can change, `robots.txt`.

LLMs results on GAIA



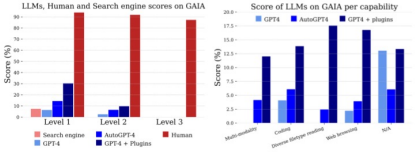


Figure 4: Left: scores per method and level. Right: scores per model and capabilities. As stated in the main text, GPT4 + plugins score should be seen as an oracle since the plugins were chosen manually. Human score refers to the score obtained by our annotators when validating the questions.

Discussion and Limitations

- **Tools unlock many use cases. More work is needed and worth it.**
- **Reproducibility issues associated with closed models and plugins/tools.**
- Static vs. dynamic benchmarks.
- Towards unified evaluation of generative models?
- Cost of designing unambiguous questions.

References

Douwe Kiela, Tristan Thrush, Kawin Ethayarajh, and Amanpreet Singh. Plotting progress in ai. *Contextual AI Blog*, 2023. https://contextual.ai/blog/plotting-progress.

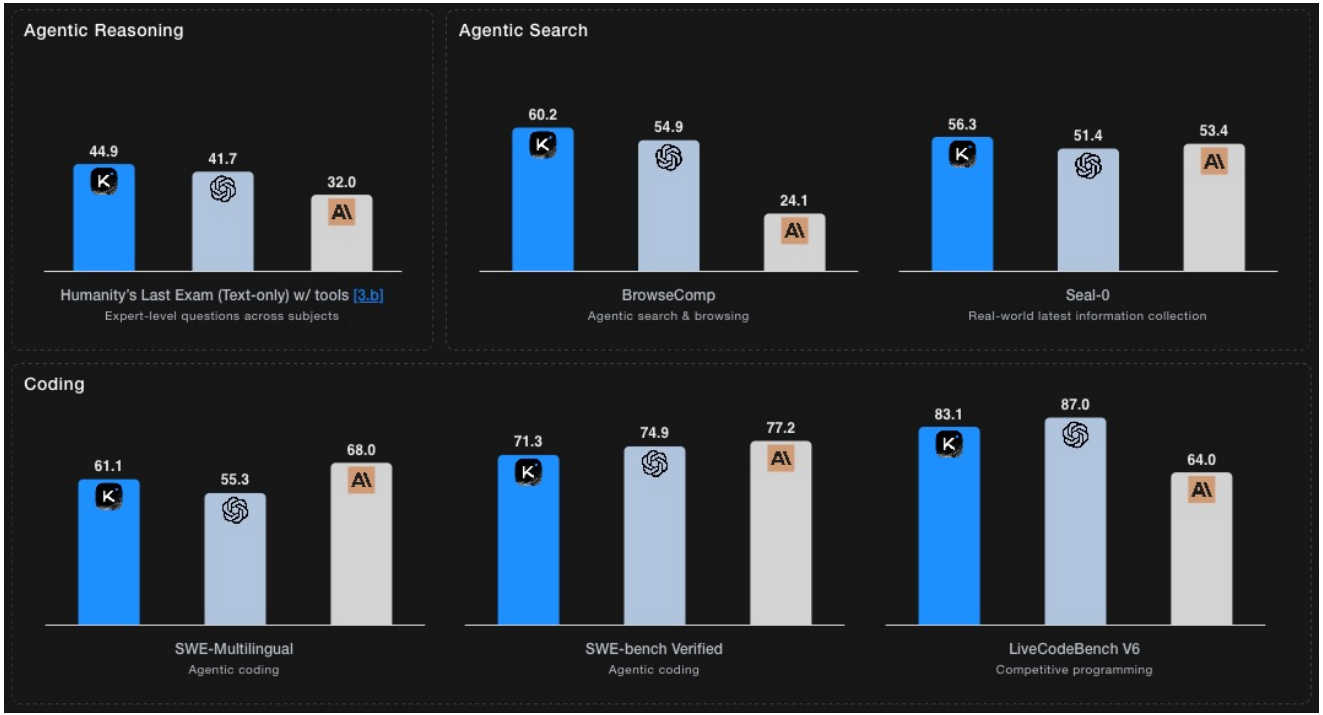
Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

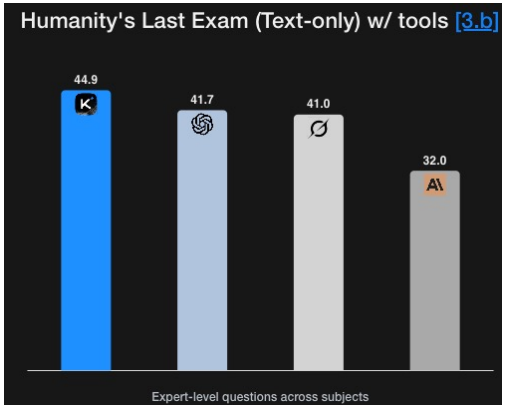
Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

Markus Jakobsson and Ari Juels. *Proofs of Work and Bread Pudding Protocols(Extended Abstract)*, pages 258–272. Springer US, Boston, MA, 1999. ISBN 978-0-387-35568-9. doi: 10.1007/ 978-0-387-35568-9 18. URL https://doi.org/10.1007/978-0-387-35568-9_18.

François Chollet. On the measure of intelligence, 2019.

Recent Progress on Coding, Agents & Computer Use w/ Kimi





https://moonshotai.github.io/Kimi-K2/thinking.html ; Release Date: Nov. 7, 2025

Latest Progress on Coding, Agents & Computer Use

	Opus 4.5	Sonnet 4.5	Opus 4.1	Gemini 3 Pro	GPT-5.1
Agentic coding SWE-bench Verified	**80.9%**	77.2%	74.5%	76.2%	76.3% 77.9% Codex-Max
Agentic terminal coding Terminal-bench 2.0	**59.3%**	50.0%	46.5%	54.2%	47.6% 58.1% Codex-Max
Agentic tool use τ2-bench	Retail **88.9%** Telecom **98.2%**	Retail 86.2% Telecom 98.0%	Retail 86.8% Telecom 71.5%	Retail 85.3% Telecom 98.0%	— —
Scaled tool use MCP Atlas	**62.3%**	43.8%	40.9%	—	—
Computer use OSWorld	**66.3%**	61.4%	44.4%	—	—
Novel problem solving ARC-AGI-2 (Verified)	**37.6%**	13.6%	—	31.1%	17.6%
Graduate-level reasoning GPQA Diamond	87.0%	83.4%	81.0%	**91.9%**	88.1%
Visual reasoning MMMU (validation)	80.7%	77.8%	77.1%	—	**85.4%**
Multilingual Q&A MMMLU	90.8%	89.1%	89.5%	**91.8%**	91.0%

https://www.anthropic.com/news/claude-opus-4-5 ;
Date: Nov. 25, 2025